COVER LETTER OF PAPER 23 ⇒ 938

Dear Meta Reviewer and Referees:

We have substantially revised the paper following the Referees' suggestions. For the convenience of Referees, changes for addressing the concerns of R1, R2 and R3 are color-coded in purple, blue and orange, respectively. We would like to thank the Referees for insightful comments and for supporting this work!

Below please find our responses to the reviews.

Response to the comments of the Meta Reviewer.

[Meta summary & recommendation]

The authors should prepare a major revision addressing all the detailed comments raised by the reviewers while paying special attention to the following concerns:

1. Writing and formal definitions: Revise Section 3, as some concepts are either not defined at all or are defined unclearly. The examples provided could also be improved (e.g., do not include the RHS in the condition, since the FD is otherwise satisfied by definition).

2. Experimental presentation: The experimental results are presented in a confusing manner because each plot refers to different combinations of datasets and algorithms. This should be revised. If only partial results are shown, the reason for choosing exactly those results should be explained.

3. Missing baselines: The experiments do not include baselines for the individual system components, e.g., a simple correlation calculation instead of the transformer or uniformly distributed sampling across all tuples for RepSampler. As a result, the concrete added value of the implementations used per component is not apparent. It would be particularly helpful to know the benefit of the transformer, as this is not a simple approach either.

4. Missing details: Add details about the resources required to train the transformer: runtime, settings, hardware, etc.

[A] Many thanks! We have addressed all the comments from the Meta Reviewer and all three Referees throughout the revised paper.

Regarding **concern 1 (writing and formal definitions)**, we have revised the preliminaries to clarify the concepts and notations in Section 3 (pp. 3–4), including the pattern tuple t_p (Definition 3.1, p. 3), the concatenation symbol "‖" (Definition 3.2, p. 3), and the scoring-based correlated attribute set $\mathbf{C}_Y$ (Definition 3.5, p. 4). We have also revised the definitions of CFD support and confidence, which decouple pattern coverage from dependency satisfaction (p. 3). Support is now defined by by the size of pattern-matching subsets, which eliminates redundancy and trivial satisfaction issues. Moreover, we have refined the running examples. Example 1 now illustrates both the syntax and semantics of CFDs without including the RHS in the constant pattern condition (p. 3). We have revised Example 2 to update the support and confidence (p. 3). We have also introduced walkthroughs for CFD minimality (Example 3, p. 4), AttrFinder (Example 4, p. 6), the theoretical sample bound (Example 5, p. 7), and RepSampler (Example 6, p. 8). Please also refer to the detailed responses to **[R1W1 & D2–D8]** for revised definitions, and **[R2W1 & D1]**, **[R2W1 & D3]** for examples.

Regarding **concern 2 (experimental presentation)**, we have reorganized the experiments to ensure consistency across all evaluations (pp. 8–12). In Table 3 (parallel scalability, p. 11), we report the execution times for both $\text{SCFDM}_{\text{part}}$ and $\text{SCFDM}_{\text{all}}$ across four datasets (RT, Census, Adult, and Syn-1). In Figure 4 (scalability evaluation, pp.10–11), we have re-plotted the subfigures using a four-column layout categorized by parameters $|r|$, $|R|$, $|\hat{\sigma}|$, $|\delta|$, and $|k|$. Each subplot now evaluates our pipelines ($\text{SCFDM}_{\text{part}}$, $\text{SCFDM}_{\text{all}}$), variants ($\text{SCFDM}_{\text{row}}$, $\text{SCFDM}_{\text{col}}$), and the four baseline algorithms (CFDMiner, CTANE, Itemset-First and FD-First) under an identical evaluation framework. Please also refer to the detailed responses to **[R1W3 & D15]** for plot restoration, **[R3W1 & D3]** for layout reordering and color assignment, **[R3W2]** for parallelizing all baseline implementations, **[R1W2 & D1]** and **[R3W1 & D1]** for the additional ablation experiments, **[R2D9]** for the newly added peak memory usage analysis, **[R1D19]** for the constant CFD discovery evaluation, and **[R1W3 & D20]** for the complete parameter sensitivity evaluation across all seven datasets.

Regarding **concern 3 (missing baselines)**, in order to verify the effectiveness of the Transformer and LSH components, we expanded the ablation studies in Exp-5 (Table 6, p. 12) on the Crop dataset by introducing five component-level baselines. To avoid timeouts, this evaluation was performed on a sample of 80,000 tuples. For AttrFinder alternatives, we replaced the Transformer with Pearson correlation, XGBoost, or Kamino. The results show that these methods identify redundant antecedent attribute sets, causing runtime bottlenecks (up to 11,025.1s for XGBoost) and dropping the F_1-score to 79.8%. For RepSampler alternatives, we replaced LSH with uniform random sampling or stratified sampling. These methods cannot sufficiently preserve the tuples essential for CFD discovery, causing the F_1-score to drop from 98.2% to 86.4% and 88.2%, respectively. This performance gap strengthens the benefits of the Transformer (which captures conditional distributions via multi-head attention) and RepSampler (which ensures similarity-aware tuple coverage with an $O(n)$ hash scan). Please also refer to the detailed responses to **[R1W2 & D1]**, **[R2D4]** and **[R3W1 & D1]**.

Regarding **concern 4 (missing details)**, for the details about resources and implementation, we have updated Section 6.2 (p. 6) and Section 9 (pp. 8–10), and added Appendices D, F, H.3, and H.4 to report the training setup and deployment strategies. Training is performed on two NVIDIA 32GB VRAM GPUs with 72GB memory allocated for preprocessing. AttrFinder embeds variables into a 128-dimensional vector space and uses a 6-layer, 4-head Transformer optimized via AdamW with a learning rate of 5×10^{-4} for up to 20 epochs and an early-stopping patience of 3 epochs. All reported runtimes for SCFDM include the processes of embedding, training, data partitioning, and rule mining. The Transformer training requires 7.42s to 23.15s (averaging 13.17s) across all workloads, which accounts for 0.54% of the total runtime on average for CFD discovery ($\text{SCFDM}_{\text{all}}$). Please also refer to the detailed responses to **[R1D13]**, **[R1D14]**, **[R2W3 & D10]** and **[R3W3]**.

Response to the comments of Referee #1.

[R1W2 & D1] *The ablation study demonstrates the impact of the two components AttrFinder and RepSampler on the final results. However, it does not show to what extent the specific algorithms proposed in this paper are responsible for these effects. In particular, the quality of the transformer-based approach for learning attribute correlations is not evaluated in isolation. A comparison with existing alternative approaches or straightforward baselines would be necessary to properly assess its contribution. In the case of AttrFinder, the transformer*

could be replaced by computing simple pairwise correlation measures. If this does not scale to the full set of tuples, the computation could be restricted to the computed sample of size N. Another existing (also deep learning–based) method for grouping attributes is presented in [77]. For RepSampler, it would be interesting to evaluate how much the final results degrade when using alternative sampling strategies. For example, one could consider a method that samples uniformly at random from the entire set of tuples and therefore does not guarantee diversity in the sample.

[A] Thanks! As suggested, we have revised Exp-5 (Table 6) by adding and evaluating all five recommended alternative configurations (p. 12), where Pearson correlation [34], XGBoost [14], and Kamino [38] as AttrFinder alternatives, and random [63] and stratified [61] sampling as RepSampler alternatives. Specifically:

(1) Replacing AttrFinder with Pearson, XGBoost, or Kamino significantly degrades discovery accuracy since these methods discover redundant antecedent attribute sets. For example, XGBoost identifies irrelevant antecedent attributes, which results in a runtime bottleneck (11, 025.1s) and reduces the F_1-score to 79.8%.

(2) Replacing RepSampler with random or stratified sampling yields limited runtime savings (less than 13s). However, this replacement reduces the F_1-score from 98.2% to 86.4% and 88.2%, respectively. These results confirm that alternative sampling strategies fail to preserve adequate tuples for CFD discovery, while AttrFinder and RepSampler together enable SCFDM to achieve both high discovery accuracy and scalability.

[R1W1 & D2] *In Definition 3.1, the introduction of the notation "||" in the pattern tuple is missing. In [26], this is defined as "We separate the attributes in X and A in a pattern tuple with '||'." A similar sentence should be added here as well.*

[A] Thanks! We have addressed this by adding Definition 3.2 (p. 3). Specifically, following [29], the concatenation symbol "||" is utilized to partition the antecedent and consequent attribute values within a pattern tuple, denoted as $t_p = (t_p[\mathbf{X}] \parallel t_p[Y])$.

[R1W1 & D3] *In other works (e.g., [24,34,75]), a pattern tableau $T_p = \{t_{p1}, t_{p2}, \dots\}$ is used to describe the condition instead of a single pattern tuple t_p (a tuple $t \in r$ satisfies the tableau T_p if it satisfies at least one pattern tuple $t_p \in T_p$). In my view, this significantly increases the expressiveness of the model, as it allows for disjunctions of atomic conditions (e.g., the FD $X \to Y$ holds if $X = A$ or $X = B$), and thus enables restricting attribute domains to arbitrary subsets. This is not possible with a single pattern tuple, where only two cases exist: for a given attribute $X_i \in X$, either exactly one value or all values of the domain are allowed. To what extent would such an abstraction affect the proposed algorithm? Would it require modifications at any stage, and if so, what consequences would this entail?*

[A] Thanks! We have revised Definition 3.3 (p. 3) to include tableau-based CFDs ($\varphi_{\mathcal{T}}$), and added Appendix B to show that SCFDM adapts to tableau-based CFDs with only minor modifications:

(1) AttrFinder requires no changes to discover tableau-based CFDs ($\varphi_{\mathcal{T}}$), as its self-attention mechanism captures the underlying conditional distribution $P_r(Y \mid \mathbf{X})$ of $\varphi_{\mathcal{T}}$ via masked token reconstruction. If a tableau-based CFD holds, multiple constant values of $\mathbf{X}$ will yield strong predictive signals for Y, which results in high attention weights for the $(\mathbf{X}, Y)$ pair.

(2) RepSampler is effective since its LSH-based partitioning assigns tuples from different constant branches (e.g., $\mathbf{X} = A$ vs. $\mathbf{X} = B$) to distinct buckets, which enables multi-round sampling to preserve representative tuples from each branch.

(3) PCFDFinder supports adaptation by either replacing the serial miner with a tableau-based method or applying a post-processing step ($O(|\Sigma|)$) to merge single-pattern CFDs that share the same attributes but differ in constant values into a tableau-based CFD.

By partitioning relations into sub-tables, SCFDM reduces the search space of tableau-based CFD miners, enabling efficient discovery of tableau-based CFDs.

[R1W1 & D4] *When using a single pattern tuple, the inclusion of the attribute Y in the pattern for an FD $X \to Y$ appears unnecessary to me. There are two cases: (1) the value $t_p[Y]$ is a variable (i.e., any value is allowed, which does not add any additional specification to the overall pattern), or (2) the value $t_p[Y]$ is a constant. In the second case, the FD is trivially always satisfied, since all tuples in the set r_ϕ have the same value in Y. A violation is therefore impossible by definition. Such patterns do not need to be discovered explicitly, but can instead be simply derived from the identified patterns over X (just combine the discovered X pattern with every possible value for Y).*

[A] Thanks! We have revised Definition 3.3 (p.3), which allows the consequent $t_p[Y]$ to be either a wildcard "_" or a constant matched by $t_p[\mathbf{X}]$. We also reviewed pattern tableau ($\mathcal{T}_p$) to merge single-pattern CFDs that share the same attributes but differ in constant values. Moreover, we added Appendix A to clarify different CFDs based on their pattern tuples (t_p).

[R1W1 & D5] *Your definition of the support of a CFD is somewhat confusing to me. In other papers such as [26,34,42], it is defined as the fraction (or number) of tuples that satisfy the given pattern ("The support of a CFD $\phi = (X \to A, t_p)$ in r, denoted by $sup(\phi, r)$, is defined to be the set of tuples t in r such that $t[X] \leq t_p[X]$ and $t[A] \leq t_p[A]$, i.e., tuples that match the pattern of ϕ." [26]). Thus, it corresponds to a form of coverage of the condition. In your definition, however, there is the additional requirement that the FD must be satisfied (i.e., "such that for any $t_1, t_2 \in r_\phi$, if $t_1[X] = t_2[X]$, then $t_1[Y] = t_2[Y]$").*

What does this imply for the support if the set r_ϕ contains one (or more) tuple pairs that violate the given FD? For example, consider the CFD ϕ: ([Gender] → Relationship, (Male ∥ _)). Would the support then be 0? In such a case, would it not make more sense to consider the size of the largest subset of r_ϕ that satisfies the FD, as done in the g_3 error measure [76]? To me, this case is not clearly defined, and it suggests two possible approaches: (1) as in [26,34,42], ignore FD satisfaction and define support purely as the number of tuples matching the pattern, or (2) as in the g_3 error, use the size of the largest subset of tuples that satisfies the FD.

[A] Thanks! We have revised the support definition in Section 3 (p. 3) to align with the existing rule discovery literature [29, 40, 49, 50], which distinguishes pattern coverage (support) from dependency satisfaction (confidence). Specifically, the support of a CFD $\varphi = (\mathbf{X} \to Y, t_p)$ is revised as the number of tuples that match the pattern t_p, i.e., $\text{supp}(\varphi, r) = |r_\varphi|$, where $r_\varphi = \{t \in r \mid t[\mathbf{X}] \leq t_p[\mathbf{X}] \wedge t[Y] \leq t_p[Y]\}$ [29]. This does not require tuples to conform with the FD when computing support. Thus, the example (i.e.,

φ : ([Gender] $\rightarrow$ Relationship, (Male $\parallel$ _))) yields a support of 5, which reflects its data coverage.

[R1W1 & D6] *I see a similar issue with the definition of confidence. In contrast to association rule mining, it seems unintuitive to relate a condition based on the attributes $X \cup Y$ to one that is based only on X. This becomes particularly problematic when aiming to express, via a high confidence value, that a CFD holds for most tuple pairs and is only violated due to a small number of data errors.*

I also have to disagree with your example for $conf(\phi_4, r)$: in my opinion, the CFD ϕ_4 holds in your example with 100% confidence. There are no tuple pairs that satisfy the pattern tuple t_p but violate the FD (for all tuples t_1, t_2 with $t_1[X \cup Y] \leq t_p[X \cup Y]$ and $t_2[X \cup Y] \leq t_p[X \cup Y]$, it holds that $t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y]$). The tuples $\{t_1, t_6\}$ are not relevant to the condition t_p at all, since they do not satisfy it. Why should they matter? Moreover, if the value $t_p[Y]$ is a variable (which, as argued above, is arguably the only meaningful case), the two sets are identical.

A short literature review shows that confidence is defined differently in other papers (e.g., [34,35,42,75]): "Definition 3: The confidence, $C^{\phi}(R)$, of a CFD ϕ on relation R is the maximum fraction of its support set that can be retained, so that after deleting all other rows, the remainder of the support set satisfies the embedded FD and all relevant assertions." [75]. This means that, similar to the g_3 error, the maximum subset of tuples that satisfies the FD is considered. Similar definitions are used in [34,35,42]. The resulting confidence thus actually reflects the relative fraction of tuples $\in r_{\phi}$ that are leading to a violation of the FD.

[A] Thanks! We have revised the confidence and Example 2 (p. 3):

(1) In Section 3 (p. 3), as suggested, we have revised confidence using the g_3-based measure [19, 40, 41, 49]. For a CFD $\varphi : (\mathbf{X} \rightarrow Y, t_p)$, given $r_{\varphi} = \{t \in r \mid t[\mathbf{X}] \leq t_p[\mathbf{X}] \wedge t[Y] \leq t_p[Y]\}$, $\text{conf}(\varphi, r)$ is revised as the maximum fraction of r_{φ} that satisfies the embedded functional dependency $\mathbf{X} \rightarrow Y$ in φ. This quantifies rule reliability in the presence of data errors and is consistent with [19].

(2) Example 2 is revised, where for the variable CFD φ_4 : ([Gender] $\rightarrow$ Relationship, (_ $\parallel$ _)), the supporting tuple set is the entire relation ($|r_{\varphi_4}| = 7$). Among the tuples satisfying Gender = Male, t_1 and t_6 take the relationship value "Not-in-family", whereas t_2, t_3, and t_5 take the value "Husband". Since these tuples share the same antecedent value but different consequent values, they violate the functional dependency. The largest conflict-free subset is $\{t_2, t_3, t_5, t_4, t_7\}$, yielding a confidence of $\frac{5}{7}$.

[R1W1 & D7] *In your definition of support, there is a redundancy. If $\{t \mid t \in r, t[X \cup Y] \leq t_p[X \cup Y]\}$ holds, then it already implies that $t[Y] \leq t_p[Y]$. The second condition is therefore unnecessary.*

[A] Thanks! We have revised the definition of support in Section 3 (p. 3), where the pattern-matching subset is revised as $r_{\varphi} = \{t \in r \mid t[\mathbf{X}] \leq t_p[\mathbf{X}] \wedge t[Y] \leq t_p[Y]\}$.

[R1D8] *"for any subset $\boldsymbol{X'} \subset \boldsymbol{X}$ ..." in the definition of the minimality of CFDs. The right-hand side X should also be in bold.*

[A] Thanks! We have corrected the right-hand side X as $\mathbf{X}$ (pp. 3–4) and maintained notation consistency throughout the paper.

[R1D9] *In Section 7, what exactly is the error bound ϵ? Does it refer to precision, i.e., the number of false positives?*

[A] Thanks! ϵ denotes the additive estimation error [6, 22] between the estimated ($\hat{\sigma}_s$) and true ($\hat{\sigma}$) normalized support, rather than precision or false positive rate. To eliminate ambiguity, we have clarified ϵ in Section 7.1 (p. 7) via the probability constraint $Pr(|\hat{\sigma}_s - \hat{\sigma}| \leq \epsilon) \geq 1 - \delta\beta$. This additive error bound enables us to leverage the variance-sensitive Hoeffding-Bernstein inequality to derive a tighter sampling lower bound.

[R1D10] *In Section 8.1, you use k in one place and l in another to denote the number of correlated attribute sets. This should be unified for consistency.*

[A] Thanks! We have revised Section 8.1 to consistently denote the number of correlated attribute sets as l (p. 8).

[R1D11] *In Table 2, I would also like to see information about the default bound K in order to better assess how many of the original tuples were actually required.*

[A] Thanks! We have updated Table 2 by introducing a column labeled "#Sampled" (p. 9). This column provides the representative tuples selected by RepSampler for each dataset (e.g., reducing the RT dataset from 123, 117 down to 8, 519 tuples).

[R1D12] *What makes the two synthetic datasets Syn-1 and Syn-2 special such that they were included? Do they address specific challenges? Were they specifically generated to target certain challenges?*

[A] Thanks! While the real-world datasets demonstrate the practical applicability of SCFDM, they do not provide verifiable ground-truth CFDs for quantitative evaluation, making it costly to establish ground-truth CFDs due to extensive expert verification and preventing controlled noise injection for robustness evaluation. Therefore, we include the synthetic datasets Syn-1 and Syn-2 in Section 9.1 (p. 8), which provide controlled environments with verifiable ground-truth CFD covers and controllable noise for evaluating both discovery accuracy and robustness.

[R1D13] *What hardware was used for training the transformer and the embeddings? I would assume that a GPU was used. What are the training hyperparameters (e.g., number of epochs, etc.)?*

[A] Thanks! We have described the hardware infrastructure and default hyperparameter configurations in Section 9.1 (p. 9). We also added Appendix H.4 to provide hyperparameter selection guidelines and Appendix D to describe the training details of AttrFinder.

(1) Hardware and environment. All experiments were conducted on a four-node cluster with a total of 224GB memory, where each node has two 28-core 2.70GHz CPUs. Transformer training was performed on two NVIDIA GPUs (32GB VRAM each). Each GPU process was allocated 72GB memory for data preprocessing.

(2) Training hyperparameters. The embedding network uses a hidden dimension of 128 and is trained with a batch size of 128. AttrFinder was optimized using AdamW with a learning rate of 5×10^{-4} for up to 20 epochs, incorporating an early-stopping patience of 3 epochs.

(3) Hyperparameter configuration guidance. To facilitate the deployment of SCFDM on new datasets, we have added Appendix H.4. This section provides a pipeline for tuning both discovery guarantee parameters (e.g., the support error bound ϵ and recall parameter β) and structural partitioning parameters (e.g., the attention retention

factor γ and significance threshold z). It also describes our adaptive parameter setting strategy, such as decreasing ϵ for large-scale datasets like Crop and Flight, to better preserve low-support CFDs.

(4) Training details. As detailed in Appendix D, AttrFinder is trained using a self-supervised masked reconstruction objective over tensor $\mathcal{T}$. During training, randomly selected attribute embeddings are replaced with a $[MASK]$ token, requiring the Transformer to predict the masked values from the remaining attributes and thereby learn conditional dependencies $P(Y \mid \mathbf{X})$. A segmented softmax layer reconstructs the categorical distribution of each attribute, and the embedding (Θ_{emb}) and Transformer (Θ_{trans}) parameters are optimized end-to-end using AdamW by minimizing the batch-averaged cross-entropy loss.

[R1D14] *What proportion of the overall runtime is spent on training the embeddings and the transformer?*

[A] Thanks! We have updated Exp-1 (Section 9.2, p. 10) and included training overhead details in Appendix F. We clarify that the reported runtime of SCFDM is end-to-end, including embedding, Transformer training, correlation extraction, sampling, and rule mining. The embedding and Transformer training overhead is small, taking only 7.42s to 23.15s (13.17s on average) across all datasets, and its proportion of total execution time scales as follows:

(1) For constant CFD discovery ($\text{SCFDM}_{\text{part}}$), training accounts for an average of 29.07% of total execution time, ranging from 57.56% on Flight, where CFD mining is relatively fast, to 20.30% on Syn-1.

(2) For both constant and variable CFD discovery ($\text{SCFDM}_{\text{all}}$), the training overhead is marginal and accounts for only 0.54% of the total runtime on average across all datasets, decreasing to 0.28% on the large Crop dataset, where it takes just 9.16s out of 3, 293.83s.

[R1W3 & D15] *In Table 3, only selected combinations of algorithm and dataset are shown. Why exactly these ones? The same applies to Figure 4: each subplot contains a different subset of approaches. What is the rationale behind this selection? This makes the results difficult to compare and creates the impression of cherry-picking in order to present the proposed approach in a more favorable light. A short explanation justifying the respective selections (and possibly adding the full plots to the extended version of the paper available in the repository) would help to dispel this concern.*

[A] Thanks! We have expanded our experimental evaluation to include full comparisons across all combinations (pp. 10–11): (1) In Table 3 (p. 11), we have extended the scalability evaluation with varying numbers of CPU cores ($|q|$), reporting execution times for both $\text{SCFDM}_{\text{part}}$ and $\text{SCFDM}_{\text{all}}$ across all four datasets (RT, Census, Adult, and Syn-1). The results show consistent parallel speedups of up to 2.52×. (2) We have re-plotted and filled in all missing curves in Figure 4 (p. 10). Every single subplot now evaluates our full pipeline ($\text{SCFDM}_{\text{part}}$, $\text{SCFDM}_{\text{all}}$), our variants ($\text{SCFDM}_{\text{row}}$, $\text{SCFDM}_{\text{col}}$), and all baselines (CTANE, CFDMiner, Itemset-First, FD-First) across all evaluated parameters ($|r|$, $|R|$, $|\hat{\sigma}|$, $|\delta|$, and $|k|$). The revised text in Section 9.2 (Exp-2, pp. 10–11) has been updated to reflect these complete speedups.

[R1D16] *In Table 4, result values for which the time limit was exceeded could be marked with an asterisk. I find it useful to distinguish whether a method is generally poor in performance or whether it simply exceeded the runtime limit.*

[A] Thanks! We have revised Table 4 and updated the discussion in Section 9.2 (Exp-3, p. 11). Specifically, (1) We have marked all intermediate metric values obtained before an algorithm reached the 9-hour execution limit with an asterisk (*), including those for CTANE on RT and for CTANE, Itemset-First, and FD-First on Syn-1. A corresponding explanatory footnote has also been added to the table. (2) We clarify that the lower F_1-scores of these baselines on large-scale datasets (e.g., Syn-1) result solely from their inability to complete execution within the 9-hour time limit due to large search space. This further highlights the efficiency of SCFDM, whose attribute correlation extraction enables efficient CFD discovery.

[R1D17] *Please bold all best-performing values in the result tables. There are several cases where results that are as good as those of SCFDM are not highlighted in bold.*

[A] Thanks! We have revised the result tables, including Table 4, Table 5, and Table 6 (pp. 11–12), as well as Table 7 and Table 8 in the Appendix, to highlight in bold the best results for each metric and any baseline results that achieve ties with SCFDM.

[R1D18] *What does the statement "which limits the max number of predicates in a CFD" mean? How can multiple predicates be contained when using a single pattern tuple? Or does this not refer to the condition?*

[A] Thanks! We have revised Section 9.2 (Exp-2, p. 10) by replacing "predicates" with "the maximum CFD size k", which constrains the total number of attributes involved in a single CFD, including both the LHS and RHS. For example, a CFD φ_3 : ([Gender, Proto] $\rightarrow$ City, (_, UDP $\parallel$ _)) has a size of $k = 3$, as it involves three attributes.

[R1D19] *In Exp-3, it would have been interesting to evaluate how well CFDMiner performs for constant CFDs. Since it is not clear how many of the CFDs are variable, and CFDMiner can only discover constant CFDs, this somewhat biases the results.*

[A] Thanks! We have added Appendix H.1 and Table 7 to focus on constant CFD discovery, comparing $\text{SCFDM}_{\text{part}}$ with CFDMiner. $\text{SCFDM}_{\text{part}}$ outperforms CFDMiner, yielding a higher average F_1-score of 0.980 compared to 0.976. On large-scale datasets (e.g., Census), CFDMiner generates spurious rules from coincidental data patterns, dragging its precision down to 0.942 (0.959 F_1). In contrast, AttrFinder and RepSampler eliminate these spurious CFDs, maintaining a high precision of 0.985 (0.977 F_1) for $\text{SCFDM}_{\text{part}}$.

[R1W3 & D20] *Why were only the two datasets RT and Syn-1 used in Exp-4? This may be justified, but the choice should be explained.*

[A] Thanks! We have added Appendix H.2 and Table 8 to report the complete parameter sensitivity results (varying $|\hat{\sigma}|$) across the remaining five datasets (Census, Crop, Flight, Adult, and Syn-2). The new data shows consistent trends. On simpler datasets (Flight and Adult), all baselines finish within the 9-hour limit and match SCFDM's optimal F_1-score. However, on large-scale datasets (Census, Crop, and Syn-2), baseline performance degrades significantly at low support thresholds ($|\hat{\sigma}| = 10^{-3}$) due to timeouts. For instance, at $|\hat{\sigma}| = 10^{-3}$, CTANE's F_1-score collapses to 0.472 on Syn-2. In contrast, SCFDM bypasses these bottlenecks via correlation extraction and representative tuple sampling, maintaining superior F_1-scores (above 0.89) across all workloads.

[R1D21] *The two references [25] and [26] are duplicates.*

[A] Thanks! We have removed the duplicate entry and updated all corresponding citations in the paper to ensure correct numbering. Furthermore, we have reviewed our entire bibliography to verify that all other references are unique and complete.

[R1W4] *The directory /parallel in the linked repository is empty.*

[A] Thanks! We identified an unintended local .git repository under the /parallel directory that prevented its files from being tracked during the initial repository push. We have resolved the issue, uploaded all source code and scripts in /parallel, and verified other project subdirectories to ensure that the repository is now complete.

Many thanks for your support and insightful suggestions!

Response to the comments of Referee #2.

[R2W1 & D1] *The presentation of the paper requires rewriting. Several terms must be formally introduced before they are used in the approach description. For example, correlating columns, pattern tuple, and the concatenation symbol ‖, . . . need explicit definitions to make the paper self-contained.*

[A] Thanks! We have restructured the preliminary (Section 3, pp. 3–4) to add explicit definitions for these concepts.

(1) Pattern tuple (Definition 3.1). We have introduced the pattern tuple t_p over $\mathbf{X} \cup \{Y\}$, specifying the legal semantics for constant assignments $a \in \mathrm{dom}(A)$ and the unnamed variable wildcard "_".

(2) Concatenation symbol (Definition 3.2). We have defined the concatenation operator "‖" to clarify how the antecedent values $t_p[\mathbf{X}]$ and consequent values $t_p[Y]$ are partitioned within CFDs.

(3) Correlated attribute set (Definition 3.5). The correlated attribute sets are now mapped onto a θ_{att}-bounded set $\mathbf{C}_Y$ using a scoring function f_{att} and a significance threshold θ_{att} derived from our Transformer's self-attention layer.

[R2W2 & D2] *The role of sub-tables and correlation is not clear in the contribution.*

[A] Thanks! We have rewritten the related work (Section 2, pp. 2–3) to show the roles and advantages of these two components.

(1) Traditional CFD discovery suffers from combinatorial bottlenecks due to exhaustive full-space column exploration. To address this, SCFDM models attribute dependencies as conditional distributions learned by a Transformer. By leveraging self-attention saliency maps, it identifies correlated attribute sets that partition the original relation into lower-dimensional sub-tables, thereby reducing the exponential search space and enabling efficient CFD discovery.

(2) Using the discovered correlated attribute sets, SCFDM partitions the original relation into lower-dimensional sub-tables, thereby decomposing the CFD mining problem into independent sub-tasks. This enables data parallelism and allows SCFDM to serve as a general framework that can be integrated with existing serial CFD mining methods, while inheriting their built-in search optimizations and pruning strategies without modifying their algorithms.

[R2W1 & D3] *While the paper uses examples, more are needed to make the paper easy to understand. Example 1 does not clearly explain what a CFD is in the given table 1 both syntactically and semantically. Furthermore, concepts such as minimality and the specific definitions in Sections 6 and 7 would benefit from dedicated walkthroughs.*

[A] Thanks! We have addressed this by revising examples:

(1) Syntax and semantics of CFDs (Example 1, p. 3). We have rewritten this example to illustrate both the syntax and semantics of CFDs. It now contrasts a constant CFD (φ_1) with variable CFDs (φ_2 and φ_3), explaining matching through constant filtering and wildcard projection, respectively, and providing concrete examples of satisfied tuples (e.g., t_2, t_6, and t_7 for φ_1) and dependency violations.

(2) Example of minimality (Example 3, p. 4). We have added an example showing the two dimensions of CFD minimality. It demonstrates why a CFD can fail to be left-reduced due to attribute redundancy (e.g., adding Gender to Workclass → Education), and why it can fail to be minimal due to pattern under-generalization (e.g., using a specific constant when it can be generalized into a wildcard "_").

(3) Walkthrough of AttrFinder (Section 6, Example 4, p. 6). We have introduced a walkthrough of our correlation extraction phase. Using concrete attention weights (e.g., 0.35 for Gender, 0.42 for Proto, and 0.04 for Age), the example formalizes the significance threshold computation ($\theta_{\text{att}} = 3/14 \approx 0.21$) to show how irrelevant attributes are pruned (e.g., $0.04 < 0.21$ for Age) and how the correlated attribute set $\mathrm{Corr}_{\text{City}} = \{\text{Gender}, \text{Proto}, \text{City}\}$ is obtained.

(4) Walkthrough of sampling bound (Section 7.1, Example 5, p. 7). To clarify our sampling theory, we added a numerical illustration of Theorem 2, showing step-by-step how the sample bound ($N \approx 37,624$) is derived via the computed threshold, and how it compares with the hybrid Chernoff bound [16], achieving more than 91.7% data reduction under $(\epsilon, \delta\beta)$ error guarantees.

(5) Walkthrough of RepSampler (Section 7.2, Example 6, p. 8). We have added a concrete trace of our representative sampling pipeline in Section 7.2. Using a target bound of $N = 5$ and an initial LSH partitioning over Table 1 ($\mathcal{B} = \{B_1 = \{t_5\}, B_2 = \{t_3, t_4\}, B_3 = \{t_1, t_2, t_6, t_7\}\}$), the example details each iteration. It illustrates how Iteration 1 extracts $\{t_5, t_3, t_2\}$ and prunes the empty bucket B_1, and how Iteration 2 selects $\{t_4, t_7\}$ to reach the target bound.

[R2D4] *It is not clear why LSH was chosen for sampling over alternatives. A comparison or justification is needed.*

[A] Thanks! Employing LSH instead of alternative sampling methods is key to balancing efficiency and accuracy by preserving tuple similarity, capturing rare patterns, and avoiding the overhead of stratified sampling. To justify this choice, we have expanded analysis in Section 7.2 (p. 7) and enriched our ablation studies in Exp-5 and Table 6 (Section 9.2, p. 12) with direct comparisons against alternative sampling methods.

(1) Theoretical analysis (Section 7.2). Compared to alternatives, LSH is well-suited to SCFDM. Uniform random sampling ignores data similarity and misses rare tuples, leading to a loss of low-support CFDs. While frequency- or value-based stratified sampling can reduce distribution bias, maintaining strata over high-dimensional, combinatorial spaces introduces computational and memory overhead. In contrast, MinHash-based LSH provides similarity-aware grouping via lightweight hashing, mapping tuples with high Jaccard similarity in a single $O(n)$ scan. By operating on non-zero indices, it preserves rare tuple patterns with low computational cost.

(2) Experimental evaluation (Exp-5 & Table 6). We evaluated alter-

native sampling methods on the Crop dataset. As shown in Table 6, replacing RepSampler with random or stratified sampling yields only limited reductions in sampling runtime (to 642.2s and 644.6s, respectively), but leads to a noticeable drop in F_1-score to 86.4% and 88.2%. This confirms that alternative sampling methods fail to preserve tuples essential for CFD discovery, whereas our LSH-driven design achieves a high 98.2% F_1-score.

[R2D5] *Table 5 does not consistently use bold text for the best-performing approach.*

[A] Thanks! We have highlighted the best-performing values in bold across the result tables, including Tables 4, 5, and 6 in the main text (pp. 11–12), as well as Tables 7 and 8 in the Appendix.

[R2D6] *The use of tabular vectorization before the Transformer stage should be better motivated in Section 4. Why is this specific embedding strategy optimal for capturing functional dependencies?*

[A] Thanks! We have revised Section 5 (p. 4) to clarify the motivation for the tabular vectorization step before the Transformer. First, discrete values in the original relation r are encoded into a zero-one matrix M, where each row encodes a tuple as a concatenation of zero-one vectors. Comparing two rows reveals attribute-wise equality (=) or inequality (≠) via the positions of the "1" tokens. Therefore, M preserves whether two tuples have the same value in each attribute, which is necessary for discovering CFDs. Furthermore, row-wise similarity within M quantifies representative tuple sampling (RepSampler), since similar tuples are more likely to support CFDs. Second, the vectors in M are mapped into dense embeddings $\mathbf{E}_i$, which are used to construct the Transformer input tensor $\mathcal{T}$. Training on $\mathcal{T}$ enables the self-attention mechanism to compare attribute representations and capture dependencies among attributes. As a result, the attention layers produce meaningful attention signals that facilitate efficient discovery of CFDs.

[R2D7] *The maximum CFD size appears to have an exponential impact on runtime. The paper should discuss if further pruning or heuristic limits can be applied to tame this.*

[A] Thanks! We have added Appendix H.3 and introduced a heuristic strategy to set the maximum CFD size.

Instead of using a fixed limit $|k|$, SCFDM adaptively determines the size k_Y for each target consequent Y. By ranking candidate antecedents according to learned attention weights $\bar{\mathbf{A}}[Y, \cdot]$, it selects the minimal subset whose cumulative weight exceeds a threshold $\gamma \in (0, 1]$, i.e., $\sum_{i=1}^{k_Y} \bar{\mathbf{A}}[Y, i] \geq \gamma$. To adaptively tune γ, we scale it based on attention entropy: $\gamma = \max\left(\gamma_0, 1 - \frac{1}{m}\sum_{i=1}^{m} \mathcal{H}(\bar{\mathbf{A}}[A_i, \cdot])\right)$, where $\gamma_0 = 0.85$ is a baseline density parameter, and $\mathcal{H}(\bar{\mathbf{A}}[Y, \cdot]) = -\frac{1}{\ln m}\sum_{k=1}^{m} \bar{\mathbf{A}}[Y, k] \ln\left(\bar{\mathbf{A}}[Y, k] + \epsilon_0\right)$ denotes the normalized Shannon entropy with $\epsilon_0 = 10^{-5}$ [79]. A low-entropy (highly concentrated) attention distribution indicates that only a few antecedents dominate the dependency signal, leading to a larger γ that retains only strongly supported antecedents. Conversely, a high-entropy (near-uniform) distribution reflects weakly differentiated attention across candidates, resulting in a smaller γ that limits the antecedent set size and mitigates combinatorial explosion under uncertain dependency patterns.

On the Adult and Crop datasets, the entropy-driven heuristic sets γ within $[0.85, 0.95]$, resulting in k_Y values bounded between 2 and 7. Compared to using a fixed limit (e.g., $k = 6$) on the Crop dataset, our heuristic strategy reduces the runtime from 1, 724.2s to 655.4s while increasing the F_1-score from 98.0% to 98.2%. These results demonstrate that our strategy prunes the search space of unpromising larger-sized CFDs while maintaining discovery accuracy.

[R2D8] *While experiments show relative robustness across parameters, a systematic approach or guidance for hyperparameter optimization is missing.*

[A] Thanks! We have added Appendix H.4 to provide guidance on hyperparameter optimization.

(1) Category 1: discovery guarantee parameters ($\epsilon, \tau, \beta, \hat{\sigma}, \delta$). These parameters determine the sample size N. Specifically, the normalized support $\hat{\sigma}$ and confidence δ are user-specified parameters that define the rule discovery problem (see problem statement in Section 3, p. 4), with typical values set to $\hat{\sigma} = 2 \times 10^{-3}$ and $\delta = 0.95$. We recommend a sequential configuration procedure: first set the recall bound $\beta \geq 0.90$, then initialize the support error as $\epsilon = 5 \times 10^{-4}$. For high-dimensional relations ($m > 50$), ϵ can be reduced to 2×10^{-4} to increase the sample size and better preserve rare tuple patterns. Finally, the Chernoff ratio τ controls bound tightness, set to $\tau = 0.05$ by default, or reduced to 0.02 when $\hat{\sigma} \leq 0.02$.

(2) Category 2: structural partitioning parameters ($\gamma, \theta_{\text{att}}, z$). These variables determine the correlated attribute sets. Users can use the entropy-driven formulation (see Appendix H.3) to compute γ. We recommend setting the significance factor z (where $\theta_{\text{att}} = z/m$) to 3, and increasing it to 4 or 5 for noisy datasets.

(3) Step-by-step optimization pipeline. Finally, we present a pipeline for new datasets. It first applies default settings ($\beta = 0.9$, $\hat{\sigma} = 2 \times 10^{-3}$, $\delta = 0.95$) with entropy-based γ, followed by a 5%–20% data slice for profiling sub-table width and the number of generated candidate CFDs. Based on the profiling results, parameters such as z and γ_0 are adjusted before running the full dataset.

[R2D9] *A memory footprint experiment is needed. Given that CFD discovery is memory-bound, documenting the peak mempry usage of the pipeline is essential.*

[A] Thanks! We have revised Exp-1 (pp. 9–10) and updated Figure 3 to analyze peak memory usage of the pipeline.

(1) Constant CFD discovery ($\text{SCFDM}_{\text{part}}$ vs. CFDMiner). In Figure 3(a) (lower), CFDMiner suffers from a large search space during rule discovery, requiring an average peak memory of 26.23GB (up to 38.85GB on Crop). In contrast, $\text{SCFDM}_{\text{part}}$ reduces this overhead to 5.11GB (5.13× reduction). Our ablation variants highlight the sources of these savings: removing the attribute correlation extraction ($\text{SCFDM}_{\text{row}}$) drives peak memory back to 19.51GB due to larger search space, while omitting sampling ($\text{SCFDM}_{\text{col}}$) increases memory to 6.56GB due to redundant tuple processing.

(2) Both constant and variable CFD discovery ($\text{SCFDM}_{\text{all}}$ vs. CTANE, Itemset-First and FD-First). Memory usage increases during discovery due to exponential growth of the search space (Figure 3(b)). Baselines require high peak memory usage, averaging 149.27 GB (CTANE), 132.48 GB (Itemset-First), and 135.33 GB (FD-First), with CTANE peaking at 195.41 GB on Crop. In contrast, $\text{SCFDM}_{\text{all}}$ limits memory consumption to an average of 12.96GB, achieving a 10.73× memory reduction compared with baselines.

These memory usage results confirm that partitioning the original relation into localized, lower-dimensional sub-tables allows SCFDM to avoid the high memory overhead of traditional CFD mining, while maintaining stable and memory-efficient execution.

[R2W3 & D10] *Does the runtime include the time taken for the training and data partitioning?*

[A] Thanks! We have updated Exp-1 (Section 9.2, p. 10) to clarify that the runtime includes both training and data partitioning overheads. A runtime analysis is provided in Appendix F (please also see our response to **[R1D14]**).

Many thanks for your support and insightful suggestions!

Response to the comments of Referee #3.

[R3W1 & D1] *The authors provide many valuable experiments. But some choice of presented information is not clear to me. E.g., for Table 3 or Figure 4, why are* $\mathrm{SCFDM}_{\mathrm{part}}$ *and* $\mathrm{SCFDM}_{\mathrm{all}}$ *evaluated on different datasets, that makes the comparison difficult. I would also like to see a discussion of Exp-5 Ablation study in comparison to* $\mathrm{SCFDM}_{\mathrm{part}}$ *vs.* $\mathrm{SCFDM}_{\mathrm{row}}$ *and* $\mathrm{SCFDM}_{\mathrm{col}}$ *where according to Figure 3a omitting RepSampler has only a comparatively small impact (*$\mathrm{SCFDM}_{\mathrm{col}}$ *vs.* $\mathrm{SCFDM}_{\mathrm{part}}$*). Also I'm wondering why the ablation study was done on a dataset, where all but the full model exceed the time limit, making an actual performance comparison difficult.*

[A] Thanks! we have revised Exp-2 (Table 3, Figure 4, pp. 10–11), and Exp-5 (Table 6, p. 12) to address all problems (please also see our response to **[R1W3 & D15]** and **[R1W2 & D1]**).

(1) We have revised Table 3 (parallelism under CPU cores $|q|$) and Figure 4 (scalability under $|r|, |R|, |\hat{\sigma}|, |\delta|, |k|$). Specifically, Table 3 presents a unified evaluation of both $\mathrm{SCFDM}_{\mathrm{part}}$ and $\mathrm{SCFDM}_{\mathrm{all}}$ across four datasets (i.e., RT, Census, Adult, and Syn-1). Figure 4 evaluates all eight algorithms under identical conditions across various scalability parameters ($|r|$, $|R|$, $|\hat{\sigma}|$, $|\delta|$, and $|k|$).

(2) To avoid the 9-hour time limit, we reran Exp-5 (Table 6) on a random sample of 80,000 tuples from the Crop dataset. This reduced setting ensures that all ablation variants complete execution, enabling full runtime and accuracy comparison. The results show that RepSampler is important for balancing efficiency and accuracy. As shown in Table 6, omitting RepSampler increases runtime from 655.4s to 3,680.8s (a 5.62× overhead) because SCFDM performs verification over all tuples. Furthermore, removing RepSampler degrades rule quality. Although exhaustive checking slightly increases recall to 98.4%, it reduces precision to 97.8% because more incorrect matches are accepted. This shows that RepSampler not only speeds up computation but also reduces incorrect CFDs in the results.

[R3W1 & D2] *Why are in table 2 the columns #Attributes and #Subtables identical?*

[A] Thanks! In Table 2, AttrFinder treats each attribute $Y \in \mathrm{attr}(\mathcal{R})$ as a consequent once and extracts a corresponding correlated attribute set $\mathbf{C}_Y$. Since each set partitions one sub-table, the number of sub-tables equals the number of attributes in the dataset.

[R3W1 & D3] *Figure 4 is confusing. Maybe reordering the subfigures into three columns and 4 rows or putting the subfigures varying the same parameter in one column would help. Additionally, I would suggest assigning individual colors to each algorithm (e.g.* $\mathrm{SCFDM}_{\mathrm{part}}$ *shouldn't be the same blue as* $\mathrm{SCFDM}_{\mathrm{all}}$*) like in Figure 3.*

[A] Thanks! We have reorganized and recolored the subfigures in Figure 4 (pp. 10–11) to improve clarity and alignment. (1) We have reorganized Figure 4 into a four-column layout to group subfigures by their target parameters: column 1 varies the sampling ratio $|r|$, column 2 varies the attribute count $|R|$, column 3 varies the support threshold $|\hat{\sigma}|$, and column 4 varies the confidence threshold $|\delta|$ and rule limit $|k|$. (2) To improve clarity, we assign distinct colors, line styles, and markers to each algorithm in Figure 4. $\mathrm{SCFDM}_{\mathrm{part}}$ and $\mathrm{SCFDM}_{\mathrm{all}}$ are further distinguished by different colors.

[R3D4] *Especially in the Proofs of Theorems 1 and 2 the equations with multiple fractions are difficult to read inline.*

[A] Thanks! We have reformatted the math by moving the derivations in Theorems 1 and 2 from inline text to display equations. The full proofs are moved to Appendix C due to page limits.

[R3W2] *If I see it correctly, none of the chosen baselines offers parallel dependency discovery. Why was none of the methods discussed in the related work section used as baseline in the experimental section?*

[A] Thanks! We have addressed this by parallelizing all baseline methods and expanding our experiments in the revised paper (pp. 8–12, please also see our response to **[R1W2 & D1]** and **[R2D4]**).

For a fair comparison between SCFDM and the baselines, we parallelized all four sequential baselines (CFDMiner, CTANE, Itemset-First, and FD-First) using the parallel lattice traversal and multi-space search strategies [30, 52, 77] in Section 2. All experiments (Exp-1 to Exp-6, pp. 8–12) and relevant Appendices (F, H) have been re-executed using these parallelized versions. Moreover, in Exp-5 (Table 6), we have incorporated and evaluated key data-driven and statistical discovery techniques reviewed in Section 2 (also see response to **[R1W2 & D1]**) as alternative baseline components. This includes replacing AttrFinder with pairwise Pearson correlation coefficient [34], XGBoost-based attribute selection [14], the data cleaning baseline Kamino [38], as well as replacing RepSampler with random [63] and stratified [61] sampling.

The empirical results show that even with parallelization or when replacing AttrFinder or RepSampler with alternative components, the baselines still experience significant accuracy degradation (F_1-score drops to 72.1%–88.2%) or severe runtime overheads(e.g., 11,025.1s for XGBoost), confirming the effectiveness of SCFDM.

[R3W3] *Description of Transformer part of the suggested approach lacks details. For example, it is mentioned that there are multiple attention heads, but I couldn't find how many. Also, information on training and inference-time would be helpful. Were GPUs used for training and inference?*

[A] Thanks! We have updated Section 6.2 (p. 6) to provide full hardware configuration details. We have added the hyperparameter configurations, including a 6-layer Transformer with 4 attention heads for tabular embedding. The Transformer training and inference are executed on two 32GB NVIDIA GPUs to accelerate global attention map extraction. As shown in Exp-1 and Appendix F, the training and inference phases are computationally efficient, requiring an average of 13.17s (ranging from 7.42 to 23.15s) across all datasets. Please also refer to our responses to **[R1D13]**, **[R1D14]**, and **[R2W3 & D10]** for more training and inference details.

Fast Discovery of Conditional Functional Dependencies via Transformer-Guided Relation Partitioning

Ruochun Jin, Shenglin Chen†, Xi Wang, Siyi Yang, Ting Wang
College of Computer Science and Technology, National University of Defense Technology, Changsha, China
jinrc@nudt.edu.cn,13272068106@163.com,18342211026@163.com,yangsi_@nudt.edu.cn,tingwang@nudt.edu.cn

ABSTRACT

Discovering conditional functional dependencies (CFDs) in large relations has been a long-standing challenge due to prohibitive computational costs. We propose SCFDM, a Transformer-guided framework that accelerates the discovery by modeling CFDs as latent conditional distributions. By projecting relational data into a high-dimensional space via a Transformer-based model, SCFDM leverages self-attention saliency to identify correlated attributes by capturing their interaction strengths. This neural-driven insight allows SCFDM to divide the exponentially large search space into independent smaller ones. To ensure discovery quality, we propose a diversity-aware sampling strategy with a theoretical lower bound for discovery accuracy, which reduces input size with performance guarantee of CFD discovery. This Transformer-guided division of the input relation enables data parallelism for any existing CFD discovery algorithm. Extensive experiments on seven real-world and synthetic datasets demonstrate that SCFDM achieves an average speedup of 105.18× (up to 136.49×) compared with baselines while maintaining high discovery accuracy of over 90%.

1 INTRODUCTION

Data dependencies, such as conditional functional dependencies (CFDs) [28], matching dependencies (MDs) [26] and functional dependencies (FDs) [18], are crucial in data management tasks, including query optimization [51], data cleaning [27], and data repair [27]. However, dependency discovery remains challenging, especially for those that contain patterns such as CFDs [30]. It is costly to discover CFDs, as it involves both identifying FDs embedded in CFDs and mining associated patterns [40], which takes $O(m \times d^m \times n)$ [72], where m, n and d are the number of attributes, tuples, and average domain size of attributes, respectively. To accelerate CFD discovery, researchers have proposed various algorithms [15, 17, 23, 24, 29, 39, 40, 46, 49, 54, 55, 72, 89, 93] with optimization strategies as follows. (1) Pruning strategies eliminate invalid candidates early to reduce the search space [29, 54, 72, 93]. (2) Tuple sampling selects subsets from the original relation to handle large datasets [8, 10, 30, 52, 57]. (3) Parallel mining employs multi-processor processing to accelerate the discovery [30, 31, 69].

However, previous methods still face the following challenges in practice. First, most existing methods discover directly on the input relation by level-wise search, where the exploration space grows exponentially with the number of attributes [29, 66]. This incurs unbearable enumeration and high memory cost when the input scales in columns. Although the support-based pruning [29–31] helps to alleviate this problem, the discovery can hardly terminate in practice when a low support threshold is set. Second, existing parallel paradigms suffer from excessive synchronization overhead as they manage interdependent search spaces [30]. As the pruning and validation of rules often depend on the state of other processors, the parallel threads frequently become idle while waiting for synchronization, resulting in high communication cost to maintain consistency across multiple cores [53]. Such dependencies among the cores make it difficult to achieve linear speedup and optimal load balancing [30]. Third, although most existing sampling methods (e.g., single-round or reservoir sampling) operate within accuracy bounds [10, 13, 30, 92], they treat each tuple equally when sampling and ignore the semantic correlations among them. Thus, these sampling methods cannot skip unrelated rows in which CFDs are unlikely to be discovered and may miss relatively rare tuples where low-support CFDs are embedded, which cause redundant search and fail to discover valid CFDs of low support.

These challenges lead to the following questions: (1) Can we capture correlations among attributes before discovery to prune the search space? (2) How can we sample tuples such that the majority of CFDs, including low-support ones, can be discovered from the sampled data? (3) Can we partition the input relation into independent sub-tables based on attribute correlations to accelerate CFD mining via data parallelism? (4) Given noisy real-world datasets, can we discover CFDs that are valid for data management tasks?

Contributions & organization. In view of these challenges, we propose SCFDM, a **S**ub-table-based **C**onditional **F**unctional **D**ependency data-parallel **M**ining framework that accelerates CFD discovery via data parallelism where the input relation is divided into independent sub-tables. Specifically, each sub-table is constructed by grouping correlated attribute sets identified by a Transformer-based model and sampling similar tuples via a representative sampling strategy. SCFDM allows any existing CFD discovery algorithms to be executed on these sub-tables in parallel, which significantly accelerates the mining process while maintaining accuracy. SCFDM has the following features.

(1) Vector representation for CFD *discovery (Section 5).* We develop a two-stage embedding pipeline that transforms raw relations into vector representations for correlation learning. We first employ one-hot encoding to convert discrete categorical values into a sparse zero-one matrix M, which is subsequently utilized to identify similar tuples that support CFDs. Vectors in M are then projected into a dense latent space to construct the input tensor $\mathcal{T}$, enabling the Transformer-based model to capture attribute-level correlations and identify the correlated attribute sets required for mining CFDs.

(2) Probabilistic modeling of CFDs *(Section 6).* We define CFDs from a statistical perspective, which formulates attribute depen-

†The corresponding author.

dencies as conditional probability distributions. By approximating these distributions, the Transformer-based model can effectively capture and learn the correlations of attributes in CFDs.

(3) Attention-driven correlated attribute set extraction (Section 6). We propose AttrFinder, a Transformer-guided module that extracts correlated attribute sets by analyzing the Transformer's global attention distributions. Instead of enumerating in the exponentially large search space, SCFDM identifies the correlations among attributes by analyzing the attention maps of the Transformer. This approach extracts highly correlated attribute sets from the learned attention weights, which are then used to partition the input relation into independent sub-tables. These sub-tables enable efficient data parallelism for existing CFD discovery algorithms.

(4) Diversity-aware representative tuple sampling (Section 7). To reduce computational overhead while providing a discovery recall guarantee, SCFDM establishes a theoretical sampling lower bound. By incorporating binomial variance into the probabilistic analysis of CFD support, SCFDM derives a tighter sample budget compared to standard Chernoff bounds. To populate this budget, we employ diversity-first selection guided by locality sensitive hashing (LSH), which partitions tuples into similarity-aware buckets. This ensures uniform sampling coverage across all data buckets, which prevents low-support CFDs from being obscured by frequent patterns while maintaining the diversity of CFDs within the sampling bound.

(5) Parallel discovery of CFDs *(Section 8).* SCFDM discovers CFDs through data parallelism guided by correlated attribute sets and representative tuples, where sampled representative tuples are projected onto correlated attribute sets to partition the original relation into smaller independent sub-tables that serve as parallel inputs.

(6) Experimental study (Section 9). Using seven real-world and synthetic datasets, we validate the following results: (a) SCFDM speeds up classical CFD mining algorithms by 105.18 times on average, while maintaining over 90% accuracy. (b) SCFDM exhibits favorable scalability as the dataset dimension grows. For example, as the number of attributes increases from 9 to 25 on the Flight dataset, its runtime rises by 31.24× only, compared to a 330.50× increase in the runtime of CTANE [29]. (c) SCFDM scales effectively with more processing cores, achieving a 2.41× speedup on the RT dataset when the number of cores increases from 28 to 112.

Novelty. The novelty of SCFDM includes: (1) a probabilistic formulation of CFDs as latent conditional distributions via a Transformer; (2) an attention-driven module for extracting correlated attribute sets and reducing the search space; (3) a diversity-aware sampling strategy with a theoretical lower bound; and (4) a data-parallel framework that constructs sub-tables via correlated attribute sets and representative sampling tuples for parallel CFD mining.

2 RELATED WORK

CFD *discovery*. Researchers have proposed various CFD discovery algorithms as follows: (1) Level-wise search with pruning is widely employed for discovering CFDs, including CTANE [29], constant-pattern-generation-based [40, 81], attribute-grid-based [15], FACD [54] and CFUN [23]. (2) Depth-first search. FastCFD [29] uses a depth-first strategy with closed-itemset pruning, while BFCFD [55] employs sampling for single-pass discovery. (3) Hybrid approaches. As shown in [72], these methods first mine patterns from itemsets and then discover FDs in the corresponding subsets, or first discover FDs and then mine patterns applicable to those FDs. (4) Others. A greedy approximation algorithm computes near-optimal tables of CFDs from embedded FDs [40]. CFDMiner employs generators and closures for efficient constant CFD discovery [29]. GCFD discovers CFDs by detecting data errors guided by user-annotated values [49]. XPlode identifies the top-ranking CFDs for data repair tasks using an on-demand approach [71].

Different from previous methods, SCFDM formulates attribute dependencies in CFDs as conditional distributions learned by a Transformer. By analyzing self-attention saliency maps, SCFDM extracts correlated attribute sets for parallel discovery. These sets partition the original relation into lower-dimensional sub-tables, effectively reducing the exponential search space into smaller ones. This Transformer-guided strategy enables fast CFD discovery via data parallelism and can be integrated with existing discovery methods.

Tuple sampling. Tuple sampling has been applied to dependency discovery [2, 9, 29, 55, 67, 68, 82, 83], including random sampling [62], stratified sampling [33], reservoir sampling [87], and focused sampling [7]. (1) Random sampling selects tuples with equal probability, where Chernoff [92] and Chebyshev [57] bounds are employed to calculate sample size based on error and confidence thresholds. (2) Stratified sampling divides data into multiple sets before sampling, which reduces the sample size based on data distribution and transactions [56, 90], where the main challenge is how to partition the strata [64]. (3) Reservoir-sampling-based CFD discovery iteratively samples data and refines the discovered rules. During this process, each tuple is evaluated to determine whether its attribute values satisfy the rules [55]. (4) Focused sampling targets at tuple pairs specified by consistency sets, evidence sets [10, 52], or errors in user-annotated attributes [49].

SCFDM differs from previous methods in that it not only samples tuples but also divides the input relation by selected columns. Specifically, based on the attention mechanism of Transformers, it reduces the search space by projecting the input onto correlated attributes where candidate CFDs are embedded. Moreover, theoretical analysis provides performance guarantee of support and confidence bounds for the sampling strategy in SCFDM.

Parallel dependency discovery. Various parallel optimizations for dependency discovery have been proposed. ParaCoDe uses a depth-first strategy with a single-node shared-everything architecture [77] to mine CFDs in parallel within a partitioned search tree, which may however suffer from node overloads [36]. Saxena et al. introduce six primitives to parallelize dependency discovery, which focus on workload allocation [47]. However, these primitives do not support CFD patterns [30]. Naumann et al. parallelize approximate dependency mining by traversing multiple search spaces [52]. Han et al. propose parallel rule discovery algorithms that ensure scalability with consideration of computational overhead [30, 31]. However, as data scales, these algorithms suffer from communication and task-allocation bottlenecks due to static coupling between search spaces and resources, limiting dynamic load balancing.

Different from previous parallel solutions [47, 52, 77], SCFDM provides a data-parallel framework that can be integrated with any existing CFD mining algorithm. By discovering CFDs within all sub-

Table 1: Example relation of adult information

ID	Name	Age	Workclass	Education	Edu-num	Relationship	Gender	Occupation	Cc	Ac	City	Proto	Service
t_1	Andy	23	Self-emp	Bachelors	13	Not-in-family	Male	Exec-managerial	05	18	LD	TCP	MQTT
t_2	Edward	34	State-gov	Bachelors	13	Husband	Male	Adm-clerical	01	33	NY	UDP	DNS
t_3	Tom	25	Private	HS-grad	9	Husband	Male	Handlers-cleaners	08	22	MS	TCP	MQTT
t_4	Angelia	33	Private	Masters	14	Wife	Female	Exec-managerial	05	18	LD	UDP	DNS
t_5	Justin	36	Self-emp	Masters	14	Husband	Male	Exec-managerial	08	22	MS	TCP	MQTT
t_6	Alex	42	State-gov	Bachelors	13	Not-in-family	Male	Prof-specialty	01	33	NY	UDP	DNS
t_7	Candice	29	State-gov	Bachelors	13	Wife	Female	Prof-specialty	01	33	NY	TCP	MQTT

tables, the rule discovery problem is deconstructed into independent search tasks. This data parallelism allows SCFDM to inherit existing well-designed search optimizations (e.g., pruning and auxiliary data structures in [29]) and the advanced performance of previous serial methods without altering their mining algorithms.

3 PRELIMINARIES

Consider a relational schema $\mathcal{R}$ defined over a set of attributes $attr(\mathcal{R})$, where each attribute $A \in attr(\mathcal{R})$ has a domain $dom(A)$. For each tuple t in relation r of $\mathcal{R}$, $t[A] \in dom(A)$, where $t[A]$ is the value of t for attribute A. Next, we review the basics of CFDs.

Definition 3.1: *As in [29], the pattern tuple t_p is defined over the attributes $\boldsymbol{X} \cup \{Y\}$, where for each attribute $A \in \boldsymbol{X} \cup \{Y\}$, $t_p[A]$ is either a constant $a \in dom(A)$ or an unnamed variable "_" that is a wildcard representing any value from $dom(A)$.*

Definition 3.2: *Following [29], we use the concatenation symbol "$\|$" to partition the antecedent and consequent attribute values within the pattern tuple, denoted as $t_p = (t_p[\boldsymbol{X}] \,\|\, t_p[Y])$*

Definition 3.3: *Given a relation r over $\mathcal{R}$, a set of attributes $attr(\mathcal{R})$, and a pattern tuple t_p, a conditional functional dependency φ is defined as:*

$$\varphi : (\boldsymbol{X} \rightarrow Y, t_p),$$

where $\boldsymbol{X}$ is a set of attributes in $attr(\mathcal{R})$, Y is an attribute in $attr(\mathcal{R})$, and $\boldsymbol{X} \rightarrow Y$ denotes the functional dependency embedded in φ. The consequent $t_p[Y]$ in the pattern tuple can either be an wildcard "_" or a constant given by the matching tuples of $t_p[\boldsymbol{X}]$. φ is a variable CFD *if t_p contains "_", and a constant* CFD *otherwise. Following [19, 28, 40], a* CFD *with pattern tableau is $\varphi_{\mathcal{T}} : (\boldsymbol{X} \rightarrow Y, \mathcal{T}_p)$, where the tableau $\mathcal{T}_p = \{t_{p_1}, \ldots, t_{p_m}\}$ is a finite set of pattern tuples. That is, multiple* CFDs *$\{(\boldsymbol{X} \rightarrow Y, t_{p_1}), \ldots, (\boldsymbol{X} \rightarrow Y, t_{p_m})\}$ that differ in their constant specifications (e.g., $\boldsymbol{X} = A$ or $\boldsymbol{X} = B$) is written as $\varphi_{\mathcal{T}}$, which can be satisfied by a tuple $t \in r$ that matches at least one $t_{p_i} \in \mathcal{T}_p$. We discuss in Appendix B about how to extend* SCFDM *to discover such tableau-based* CFDs *with few modifications.*

Semantics of CFDs. Following [29], an order $\leq$ on constants and the unknown variable "_" is defined as follows: $\eta_1 \leq \eta_2$ if $\eta_1 = \eta_2$, or η_1 is a constant and η_2 is "_". The order $\leq$ naturally extends to tuples, e.g., $(\text{Male}, \text{UDP} \,\|\, \_) \leq (\_, \text{UDP} \,\|\, \_)$. $t_{p1}[\mathbf{X}] \ll t_{p2}[\mathbf{X}]$ if $t_{p1}[\mathbf{X}] \leq t_{p2}[\mathbf{X}]$ but $t_{p2}[\mathbf{X}] \not\leq t_{p1}[\mathbf{X}]$, i.e., t_{p2} is more general than t_{p1}. For example, $(\text{Male}, \text{UDP} \,\|\, \_) \ll (\_, \text{UDP} \,\|\, \_)$. Given a CFD $\varphi : (\mathbf{X} \rightarrow Y, t_p)$ and an instance r of $\mathcal{R}$, we say that r satisfies φ, denoted by $r \models \varphi$, iff for each pair of tuples t_1, t_2 in r, if $t_1[\mathbf{X}] = t_2[\mathbf{X}] \leq t_p[\mathbf{X}]$, then $t_1[Y] = t_2[Y] \leq t_p[Y]$.

Example 1: *We illustrate the syntax and semantics of both constant and variable* CFDs *based on tuples in Table 1.*

Syntax: Consider the following three CFDs *defined over Table 1: $\varphi_1 : ([\text{Workclass}] \rightarrow \text{Education}, (\text{State-gov} \,\|\, \text{Bachelors}))$, $\varphi_2 : ([\text{Education}] \rightarrow \text{Edu-num}, (\_ \,\|\, \_))$, and $\varphi_3 : ([\text{Gender}, \text{Proto}] \rightarrow \text{City}, (\_, \text{UDP} \,\|\, \_))$. φ_1 is a constant* CFD *as its pattern tuple contains only specific constants. It dictates that the functional dependency Workclass → Education is restricted to the pattern (State-gov ‖ Bachelors). φ_2 and φ_3 are variable* CFDs *since they contain the wildcard "_" that can take any value in the attribute domains.*

Semantics: A relation conforms with a CFD *if any pair of tuples matching the antecedent pattern adheres to the consequent pattern. For the constant* CFD *φ_1, we filter tuples where Workclass = State-gov (i.e., t_2, t_6, t_7). Since all of them have Education = Bachelors, Table 1 satisfies φ_1. For the variable* CFD *φ_3, it applies to all tuples whose Proto = UDP regardless of their gender and city (due to the wildcard "_"). The matching tuples are t_2, t_4, t_6, all of which satisfy the embedded* FD *$[\text{Gender}, \text{Proto}] \rightarrow \text{City}$ in φ_3, meaning Table 1 satisfies φ_3.*

By contrast, Table 1 does not satisfy the variable CFD *$\varphi : ([Cc] \rightarrow \text{Proto}, (\_ \,\|\, \_))$, as t_1 and t_4 violate its semantic. Specifically, although both tuples project to the same value on the antecedent attribute ($t_1[Cc] = t_4[Cc] = 05$), they map to conflicting values on the consequent attribute ($t_1[\text{Proto}] = \text{TCP} \neq t_4[\text{Proto}] = \text{UDP}$).*

Support. Following previous CFD discovery methods [29, 40, 49, 50], the support of a CFD $\varphi = (\mathbf{X} \rightarrow Y, t_p)$ in a relation r, denoted by $\text{supp}(\varphi, r)$, is defined as the number of tuples that match the pattern t_p. Specifically, let $r_\varphi = \{t \in r \mid t[\mathbf{X}] \leq t_p[\mathbf{X}] \wedge t[Y] \leq t_p[Y]\}$, the support of φ is given by $\text{supp}(\varphi, r) = |r_\varphi|$. φ is *σ-frequent* if $\text{supp}(\varphi, r) \geq \sigma$, where σ is the support threshold.

Confidence. We adopt the g_3-based confidence measure [19, 40, 41, 49] to evaluate CFD reliability in the presence of data inconsistencies. Let $r_\varphi = \{t \in r \mid t[\mathbf{X}] \leq t_p[\mathbf{X}] \wedge t[Y] \leq t_p[Y]\}$. The confidence of φ on r, denoted by $conf(\varphi, r)$, is the maximum fraction of tuples in r_φ that satisfy the embedded FD $\mathbf{X} \rightarrow Y$ [19]. Formally, $conf(\varphi, r) = \frac{\max\{|s| \mid s \subseteq r_\varphi \wedge s \models (\mathbf{X} \rightarrow Y)\}}{|r_\varphi|}$. A CFD φ is δ-confident if $conf(\varphi, r) \geq \delta$, where δ is the confidence threshold.

Definition 3.4: *Following Definition 3.3, given the support and confidence thresholds σ and δ, and a relation r, a* CFD *φ over r that satisfies σ and δ is defined as:*

$$\varphi : (\boldsymbol{X} \rightarrow Y, t_p), s.t.\ \text{supp}(\varphi, r) \geq \sigma \wedge \text{conf}(\varphi, r) \geq \delta.$$

The confidence threshold helps to mine CFDs in real-life data with noise, where CFDs that violate a few noisy tuples can still be discovered. Meanwhile, the support threshold filters out trivial or complex CFDs that overfit noisy data [91], since support is anti-monotonic and prunes invalid CFDs during rule discovery [29].

Example 2: *Continue with Example 1, given a* CFD *$\varphi_4 : ([\text{Gender}] \rightarrow \text{Relationship}, (\_ \,\|\, \_))$, we have $\text{supp}(\varphi_2, r) = \text{supp}(\varphi_4, r) = 7$ (supported by all tuples in r), $\text{supp}(\varphi_1, r) = 3$ (supported by $\{t_2, t_6, t_7\}$), $\text{conf}(\varphi_3, r) = 1$ (supported by $\{t_2, t_4, t_6\}$ without any violations), and $\text{conf}(\varphi_4, r) = \frac{5}{7}$ (violated by $\{t_1, t_6\}$).*

Minimality of CFDs. A CFD $\varphi : (\mathbf{X} \rightarrow Y, t_p)$ over r is *nontrivial* if $Y \notin \mathbf{X}$. Under the support (σ) and confidence (δ) thresholds, (1) a constant [29] CFD $\varphi : (\mathbf{X} \rightarrow Y, t_p)$ is *left-reduced* over r if for

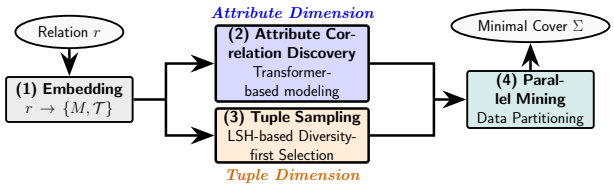


Figure 1: Overview of SCFDM

any subset $\mathbf{X}' \subset \mathbf{X}$, $r \not\models (\mathbf{X}' \to Y, t_p)$; (2) a variable [29] CFD $\varphi : (\mathbf{X} \to Y, t_p)$ is *left-reduced* over r if $r \not\models (\mathbf{X}' \to Y, t_p)$ for any subset $\mathbf{X}' \subset \mathbf{X}$, and $r \not\models (\mathbf{X} \to Y, t_p')$ for any t_p' with $t_p \ll t_p'$. A *minimal* CFD φ over r is *nontrivial* and *left-reduced*.

Example 3: *We illustrate the minimality using Table 1. First, consider a constant* CFD φ_c : $([Workclass, Gender] \to Education, (State\text{-}gov, Male \parallel Bachelors))$. *While φ_c holds, it is not left-reduced (hence non-minimal) as the attribute Gender is redundant; the subset $\mathbf{X}' = \{Workclass\}$ forms the minimal rule φ_1 in Example 1. Second, consider a variable* CFD φ_v : $([Education] \to Edu\text{-}num, (Bachelors \parallel 13))$. *Although φ_v is nontrivial, it is not minimal since its pattern can be generalized [29] by replacing the constant with a wildcard, yielding a more general minimal rule φ_2 : $([Education] \to Edu\text{-}num, (\_ \parallel \_))$ that covers all tuples.*

Cover of CFDs. Consider a set Σ of *minimal* CFDs on r [30]. We say that Σ *entails* a CFD φ, denoted by $\Sigma \models \varphi$, if $r \models \Sigma$ deduces $r \models \varphi$. Σ is *equivalent* to another set Σ', denoted by $\Sigma \equiv \Sigma'$, if $\Sigma \models \varphi'$ for any $\varphi' \in \Sigma'$ and vice versa. Σ is *minimal* if for each CFD $\varphi \in \Sigma$, Σ is not *equivalent* to $\Sigma \setminus \varphi$, i.e., each CFD φ in Σ is *non-redundant*. A cover Σ_c of Σ on r is a subset of Σ such that (1) $\Sigma_c \equiv \Sigma$, (2) all rules φ in Σ_c are *minimal*, and (3) Σ_c is minimal, i.e., Σ_c includes no *redundant* rules.

Definition 3.5: *Given a consequent attribute $Y \in attr(\mathcal{R})$ and a correlation scoring function f_{att}, the correlated attribute set $\mathbf{C}_Y \subseteq attr(\mathcal{R})$ with respect to Y is defined as: $\mathbf{C}_Y = \{Y\} \cup \{X \in attr(\mathcal{R}) \setminus \{Y\} \mid f_{att}(X, Y) \geq \theta_{att}\}$, where θ_{att} is defined as a structural significance threshold in Section 6.3. This set $\mathbf{C}_Y$ specifies the attribute partitioning for constructing the sub-tables*

Problem statement. Given a noisy relation r of schema $\mathcal{R}$, support and confidence thresholds σ and δ, the CFD discovery problem is to identify a minimal cover Σ consisting of all minimal CFDs over r that satisfy the thresholds σ and δ.

4 SOLUTION OVERVIEW

As shown in Figure 1, SCFDM takes a relation r as input and outputs a minimal cover of CFDs. Each module operates as follows.

Relational encoding and attribute embedding. This module converts relation r into vector representations via a two-stage process (Section 5). It first maps discrete values to a zero-one matrix M to construct the high-dimensional Transformer input tensor $\mathcal{T}$ and facilitate representative tuple sampling. Then, vectors in M are projected into dense latent states $\mathbf{E}_i$ in $\mathcal{T}$, enabling the dot-product self-attention mechanism to measure attribute-wise correlations in r. Furthermore, row-wise similarity within M quantifies representative tuple sampling, as similar tuples are more likely to support CFDs. This embedding strategy is well-suited for CFD discovery. First, each row of matrix M encodes a tuple as a concatenation of zero-one vectors. Comparing two rows reveals whether the corresponding attribute values are equal or different: matching one positions indicate equality (=), while different one positions indicate inequality (≠). Therefore, M preserves whether attribute values are equal or different across tuples, which is necessary for evaluating CFDs. Second, the subsequent tensor $\mathcal{T}$ provides the structured input for the Transformer to learn attribute correlations. By training on $\mathcal{T}$, the Transformer's self-attention layers capture cross-domain dependencies, generating informative attention patterns that serve as direct signals for efficient CFD discovery.

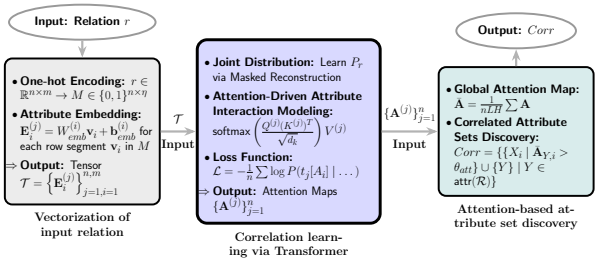


Figure 2: Pipelines of Transformer-based model

Correlated attribute sets discovery. This module identifies attribute dependencies by modeling a CFD as a conditional probability distribution (Section 6). It employs a Transformer encoder to learn joint distributions through masked value reconstruction [21, 44], capturing attribute correlations via the self-attention mechanism. Based on the global attention maps in Transformer [86], it takes $O(m^2)$ for the module to identify candidate antecedent sets $\mathbf{X}$ for each consequent Y. These identified correlated attribute sets facilitate data parallelism by guiding the partition of the relation into independent sub-tables for accelerated CFD discovery.

Representative tuple sampling. This module extracts representative tuples from r with a discovery recall guarantee (Section 7). It first derives a sampling lower bound N, which reduces sample size by incorporating binomial variance. To preserve discovery recall, this module employs similarity-aware stratified sampling via locality sensitive hashing (LSH), partitioning tuples into similarity buckets in linear time. By adopting a diversity-first selection logic, it ensures uniform coverage across all data buckets and prevents low-support CFDs from being obscured by dominant patterns, thereby maintaining the diversity of CFDs within the sampling budget N.

Parallel discovery of CFDs**.** SCFDM projects the sampled representative tuples to multiple correlated attribute sets and produces sub-table inputs for CFD discovery. Then, SCFDM employs existing discovery algorithms to find CFDs from these independent sub-tables in parallel. That is, SCFDM can parallelize any CFD discovery algorithm via data parallelism (Section 8).

Specifically, Figure 2 illustrates the pipeline of the Transformer-based model in SCFDM that takes relation r as input and yields its correlated attribute sets $Corr$. This workflow encompasses relation vectorization (Section 5), correlation learning via Transformer (Section 6.2), and attention-based attribute set discovery (Section 6.3). The process begins with vectorization, transforming relation r over $attr(\mathcal{R})$ into a high-dimensional tensor $\mathcal{T} \in \mathbb{R}^{n\times m\times d}$ by applying one-hot encoding into a zero-one matrix $M \in \{0,1\}^{n\times\eta}$, which an embedding layer then projects into $\mathcal{T}$. This tensor $\mathcal{T}$ inputs to the learning module, where a Transformer encoder performs masked attribute modeling. By minimizing reconstruction loss $\mathcal{L}$ across all tuples, the model learns the conditional probability distribution $P_r(A_i \mid attr(\mathcal{R}) \setminus \{A_i\})$ and generates attention maps $\{\mathbf{A}^{(j)}\}$ capturing attribute correlations. Finally, the module performs global anal-

ysis on these maps to construct a global map $\bar{\mathbf{A}}$, which is iteratively leveraged to identify correlated antecedents $\mathbf{X}$ exceeding a significance threshold for each attribute Y, yielding $Corr$ in $O(m^2)$ time.

5 VECTORIZATION OF INPUT RELATION

This section presents a two-stage vectorization of relation r, where one-hot encoding produces a binary matrix M (Section 5.1) and an attribute embedding layer projects M into tensor $\mathcal{T}$ (Section 5.2). This representation enables the Transformer to model attribute correlations in CFDs via dot-product similarities in $\mathcal{T}$.

5.1 One-hot Encoding of Tuples

Given a relation r over the attribute set $attr(\mathcal{R}) = \{A_1, \ldots, A_m\}$, let $dom(A_i)$ denote the domain of each attribute A_i with $c_i = |dom(A_i)|$. Each value $a \in dom(A_i)$ is encoded as a zero-one vector $\mathbf{v}_i \in \{0,1\}^{c_i}$, where only the j-th element x_j is set as 1 if a is the j-th value (alphabetically ordered) in $dom(A_i)$. The representation of each tuple $t_j \in r$ is the concatenation of zero-one vectors $\mathbf{m}_j = [\mathbf{v}_1^\top, \mathbf{v}_2^\top, \ldots, \mathbf{v}_m^\top]$ that corresponds to its m attribute values (ordered alphabetically by attribute names in $attr(\mathcal{R})$), where $\mathbf{m}_j \in \{0,1\}^{1\times\eta}$ and $\eta = \sum_{i=1}^{m} c_i$. Thus, the relation r is transformed into a zero-one valued matrix $M \in \{0,1\}^{n\times\eta}$ by encoding each tuple as a zero-one vector.

5.2 Embedding of Attributes and Relations

Given the zero-one encoding matrix M (Section 5.1), we design an attribute embedding layer (Figure 2) for Transformer-based attribute correlation modeling [85]. Specifically, for a row $\mathbf{m}_j = [\mathbf{v}_1^\top, \mathbf{v}_2^\top, \ldots, \mathbf{v}_m^\top]$ in M, each segment $\mathbf{v}_i \in \{0,1\}^{c_i}$ $(1 \le i \le m)$ is projected to a d-dimensional vector via an embedding layer (similar to [45]): $\mathbf{E}_i^{(j)} = W_{emb}^{(i)}\mathbf{v}_i + \mathbf{b}_{emb}^{(i)}$, where $\mathbf{E}_i^{(j)} \in \mathbb{R}^d$ represents the embedding of attribute A_i in tuple t_j. Thus, a tuple $t_j \in r$ is transformed into a matrix $\mathbf{H}_{in}^{(j)} = [\mathbf{E}_1^{(j)}, \ldots, \mathbf{E}_m^{(j)}]^\top \in \mathbb{R}^{m\times d}$ and we denote the trainable weights of the attribute embedding layer as $\Theta_{emb} = \{W_{emb}^{(i)}, \mathbf{b}_{emb}^{(i)}\}_{i=1}^{m}$ (weight details refer to Section 6.2 and Appendix D in [1]). These parameters Θ_{emb} are randomly initialized and optimized with the downstream Transformer (Section 6). By embedding all n rows in M, the attribute embedding layer produces a tensor $\mathcal{T} \in \mathbb{R}^{n\times m\times d}$, which enables the Transformer to treat the relation as a sequence of m attribute embeddings for each of the n tuples. This vectorized sequence enables the Transformer to capture attribute correlations via self-attention.

Remark. Modeling relation r as dense embeddings $\mathbf{E}_i$ enables the discovery of attribute correlations. Self-attention in a uniform d-dimensional space computes dot-product similarities between attributes, making hidden CFD correlations measurable via embeddings and quantifying the relationship between a candidate antecedent set $\mathbf{X}$ and a consequent Y.

6 CORRELATED ATTRIBUTE DISCOVERY

In this section, we first formulate CFDs as conditional probability distributions (Section 6.1). Based on this formulation, we train a Transformer model using the embedding matrix $\mathcal{T}$ derived in Section 5.2 to learn the attribute correlations by modeling these conditional distributions (Section 6.2). Finally, we demonstrate how the attention maps within the trained Transformer can be utilized to capture and identify correlated attribute sets in CFDs (Section 6.3).

Algorithm 1 AttrFinder (Attention-based Discovery)

Input: Relation r, transaction matrix $M \in \{0,1\}^{n\times\eta}$, mask probability p_{mask}, attention threshold θ_{att}
Output: Correlated attribute sets $Corr$
$Corr \leftarrow \emptyset$
while not converged **do**
  $\mathcal{B} \leftarrow \text{SampleBatch}(M)$
  $\tilde{\mathcal{B}} \leftarrow \text{ApplyMask}(\mathcal{B}, p_{mask})$
  $\mathcal{T}_{\mathcal{B}} \leftarrow \text{Embedding}(\tilde{\mathcal{B}})$
  $\hat{Y} \leftarrow \text{TransformerEncoder}(\mathcal{T}_{\mathcal{B}})$
  $\mathcal{L} \leftarrow \frac{1}{|\mathcal{B}|}\sum_{j\in\mathcal{B}} \text{CrossEntropy}(M_j, \hat{Y}_j)$
  UpdateWeights($\mathcal{L}$)
$\bar{\mathbf{A}} \leftarrow \frac{1}{n\cdot L\cdot H}\sum_{j=1}^{n}\sum_{l=1}^{L}\sum_{h=1}^{H}\mathbf{A}_{j,l,h}$
for each attribute $Y \in attr(\mathcal{R})$ **do**
  $\mathbf{Row}_Y \leftarrow \bar{\mathbf{A}}[Y, \cdot]$
  $\theta_{att} \leftarrow k/m$
  $\mathbf{X}_Y \leftarrow \{A_i \mid \bar{\mathbf{A}}[Y,i] \ge \theta_{att}, A_i \ne Y\}$
  if $\mathbf{X}_Y$ is not empty **then**
    $Corr_Y \leftarrow \mathbf{X}_Y \cup \{Y\}$
    $Corr \leftarrow Corr \cup \{Corr_Y\}$
return $Corr$

6.1 CFD as Conditional Probability

Inspired by the statistical analysis of FDs in [91], we first analyze CFDs from a statistical perspective. Following [35, 91], consider a tuple set $r_\varphi \subseteq r$, where $r_\varphi = \{t \in r \mid t[X] \le t_p[X] \wedge t[Y] \le t_p[Y]\}$, and for any two tuples $t_i, t_j \in r_\varphi (i \ne j)$, if $t_1[\mathbf{X}] = t_2[\mathbf{X}]$, then $t_1[Y] = t_2[Y] \le t_p[Y]$. A CFD $\varphi : (\mathbf{X} \to Y, t_p)$ is a statement over the attribute set $\mathbf{X} \subseteq attr(\mathcal{R})$ and an attribute $Y \in attr(\mathcal{R})$, where an assignment to $\mathbf{X}$ uniquely determines the value of Y on the subset r_φ above support and confidence thresholds. Thus, $\wedge_{A\in\mathbf{X}} t_i[A] = t_j[A]$ and $t_i[Y] = t_j[Y]$ can be viewed as two associated events, and a CFD can be written as the conditional probability of event $t_i[Y] = t_j[Y]$ given $\wedge_{A\in\mathbf{X}} t_i[A] = t_j[A]$ as the condition.

Definition 6.1: *Given a* CFD $\varphi : (\boldsymbol{X} \to Y, t_p)$*, a tuple set* $r_\varphi \subseteq r$ *that conforms with* φ*, the conditional probability representation of* φ *is* $\forall t_i, t_j \in r_\varphi : P_r(t_i[Y] = t_j[Y] | \wedge_{A\in\boldsymbol{X}} t_i[A] = t_j[A]) \ge \delta$.

Here, $P_r(t_i[Y] = t_j[Y] | \wedge_{A\in\mathbf{X}} t_i[A] = t_j[A]) \ge \delta$ is the conditional probability between $\mathbf{X}$ and Y restricted to r_φ. Consider the CFD φ in Definition 6.1, where $\mathbf{X} = \{A_1, A_2, \cdots, A_k\}$ is a multi-variable set and Y is a variable. The domain of $\mathbf{X}$, denoted as $dom(\mathbf{X})$, is determined by r_φ, i.e., $dom(\mathbf{X}) = \{t_i(A_1) \times t_i(A_2) \times \cdots \times t_i(A_k)\}$ for all $t_i \in r_\varphi$. When $\mathbf{X}$ is assigned by values in $dom(\mathbf{X})$, there exists a function of conditional association $Y = f(\mathbf{X})$ from $\mathbf{X}$ to Y such that if $y = f(\mathbf{x})$ then $\forall \mathbf{x} \in dom(\mathbf{X}) : P_r(Y = y | \mathbf{X} = \mathbf{x}) \ge \delta$. That is, when $\mathbf{X}$ takes values in its domain $dom(\mathbf{X})$, Y corresponds to a value $f(\mathbf{X})$ with at least δ probability, as constrained by $\forall t_i, t_j \in r_\varphi : P_r(t_i[Y] = t_j[Y] | \wedge_{A\in\mathbf{X}} t_i[A] = t_j[A]) \ge \delta$.

6.2 Correlation Learning via Transformer

In order to learn the conditional probability of CFDs, we formulate an end-to-end reconstruction task for the Transformer, where correlations among attributes are captured by learning the conditional probability $\forall t_i, t_j \in r_\varphi : P_r(t_i[Y] = t_j[Y] | \wedge_{A\in\mathbf{X}} t_i[A] = t_j[A]) \ge \delta$. Specifically, we treat each tuple vector in $\mathcal{T}$ as a sequence and train the Transformer model to recover masked at-

tribute values, thereby completing the learning of conditional probabilities in CFDs driven by this representation reconstruction task.

For each tuple t_j, each attribute $A_i \in \text{attr}(\mathcal{R})$ is iteratively masked with a [MASK] token. An L-layer Transformer applies self-attention over the remaining $m-1$ attributes to obtain a hidden representation that captures attribute correlations: $\text{Attention}(Q^{(j)}, K^{(j)}, V^{(j)}) = \text{softmax}\left(\frac{Q^{(j)}(K^{(j)})^T}{\sqrt{d_k}}\right)V^{(j)}$, where $Q^{(j)}, K^{(j)}, V^{(j)}$ are linear projections of the attribute embeddings $\mathbf{H}_{in}^{(j)}$ in $\mathcal{T}$. Notably, $\mathbf{A}^{(j,l,h)} = \text{softmax}(Q^{(j,l,h)}(K^{(j,l,h)})^T/\sqrt{d_k})$ is the self-attention map that quantifies attribute dependencies. Stacking L layers ($L = 6$ as commonly used for tabular data [41, 44]) enables iterative refinement of these dependencies, capturing attribute correlations (e.g., X_1 and X_2 jointly determining Y). Specifically, the multi-head self-attention uses four attention heads.

The attribute embedding layer and the Transformer model are optimized by minimizing the average cross-entropy loss across all tuples:

$$\mathcal{L}(\Theta) = -\frac{1}{n}\sum_{j=1}^{n} \log P(t_j[A_i] \mid t_j[\text{attr}(\mathcal{R}) \setminus \{A_i\}]; \Theta),$$

which forces the Transformer to learn the conditional probability distribution P_r of relation r, where $\Theta = \{\Theta_{\text{emb}}, \Theta_{\text{trans}}\}$ is iteratively optimized including the attribute embedding parameters Θ_{emb} and the Transformer parameters Θ_{trans}. By minimizing $\mathcal{L}(\Theta)$, the Transformer maximizes the likelihood of predicting the true attribute values, thereby aligning its internal attention mechanism with the conditional distributions P_r of the CFDs. This optimization compels the Transformer to discover attribute correlations by focusing on attributes that provide the relatively stronger predictive signals for the masked targets. The parameters $\Theta = \{\Theta_{emb}, \Theta_{trans}\}$ are adjusted through backpropagation using the AdamW optimizer [59]. The optimization and training phase are executed on two 32GB NVIDIA GPUs. Training details refer to Appendix D in [1].

6.3 Attention-based Attribute Set Discovery

The extraction of correlated attribute sets in CFDs is achieved through the analysis of attention maps in the trained Transformer of Section 6.2. As shown in Algorithm 1, AttrFinder takes two steps to identify correlated attribute sets.

Step 1: Attention map extraction (Lines 4–11). Following the optimization process described in Section 6.2 (Lines 4–10), this optimization process compels the attention maps $\{\mathbf{A}^{(j)}\}_{j\in\mathcal{B}}$ to quantify the attribute correlations required to model the conditional probabilities of CFDs. Then, AttrFinder computes the global attention map $\bar{\mathbf{A}} \in \mathbb{R}^{m\times m}$ by averaging the attention maps across all n tuples, L layers, and H heads (Line 11):

$$\bar{\mathbf{A}} = \frac{1}{n\cdot L\cdot H}\sum_{j=1}^{n}\sum_{l=1}^{L}\sum_{h=1}^{H}\mathbf{A}^{(j,l,h)},$$

where each entry $\bar{\mathbf{A}}_{i,k}$ quantifies the correlation between attribute A_i and A_k. A higher value of $\bar{\mathbf{A}}_{i,k}$ indicates a stronger conditional probability P_r in CFDs.

Step 2: Correlated attribute sets identification (Lines 12–19). AttrFinder utilizes the global attention map $\bar{\mathbf{A}}$ in Step 1 to identify correlated attribute sets of CFDs. By iteratively treating each attribute $A_i \in \text{attr}(\mathcal{R})$ as a consequent Y (Line 12), AttrFinder retrieves the row corresponding to attribute Y from the global attention map $\bar{\mathbf{A}}$ (Line 13), which captures the cumulative influence of all other attributes on Y. We then determine the candidate antecedent set $\mathbf{X}$ by selecting attributes whose value of $\bar{\mathbf{A}}_{i,k}$ exceed a threshold θ_{att}. Following common practice in attention analysis [20, 86], we set $\theta_{\text{att}} = \frac{z}{m}$ (Line 14), where $z > 1$ is a scaling factor, filtering out attributes with negligible predictive contribution relative to a uniform distribution of $1/m$. For each consequent attribute Y, the resulting correlated attribute set is $Corr_Y = \{X_i \mid \bar{\mathbf{A}}_{Y,i} > \theta_{att}\} \cup \{Y\}$ (Lines 15–17). The collection of correlated attribute sets for relation r is obtained by iterating over all attributes in attr($\mathcal{R}$): $Corr = \{Corr_Y \mid Y \in attr(\mathcal{R})\}$ (Line 18).

Example 4: *We provide a walkthrough of* AttrFinder *using Table 1, which discovers the correlated attribute set for the consequent Y = City, conforming to the conditional probability formulation in Definition 6.1. After optimization, the weights in the global attention map $\bar{\mathbf{A}}$ reflect the learned conditional distributions P_r. For the row of City, suppose the weights are $\bar{\mathbf{A}}$[City, Gender] = 0.35, $\bar{\mathbf{A}}$[City, Proto] = 0.42, and $\bar{\mathbf{A}}$[City, Age] = 0.04.*

Given $m = 14$ and $k = 3$, the threshold is $\theta_{att} = 3/14 \approx 0.21$. According to Step 2, as the scores for Gender and Proto exceed θ_{att}, they indicate a strong conditional dependency with respect to City, whereas Age ($0.04 < 0.21$) falls below the threshold and is pruned. This extracts the correlated attribute set $Corr_{City} = \{Gender, Proto, City\}$, establishing the context required for sub-table partitioning.

Complexity. AttrFinder's complexity is $O(n\cdot m^2)$ for n tuples and m attributes; other factors (epochs, layers, and embedding dimension) are treated as constants (see Appendix E in [1] for details).

7 SAMPLING WITH ACCURACY BOUNDS

Mining CFDs on massive datasets remains costly as the number of tuples grows. To avoid rule loss caused by improper sampling [30], we propose RepSampler, an accuracy-guaranteed sampling algorithm for efficient CFD discovery. We first derive a sampling lower bound (Section 7.1), and then select representative tuples from stratified partitions to preserve CFD diversity (Section 7.2).

7.1 Lower Bound for Sampling

To mitigate CFD loss during tuple sampling, we investigate the lower bound of the sample size with accuracy guarantees. Let Σ be the ground-truth CFDs holding on the original relation r, and Σ_s be the CFDs discovered from the sampled relation r_s. We evaluate discovery quality via precision and recall: $\text{precision}(\Sigma, \Sigma_s) = \frac{|\Sigma_s \cap \Sigma|}{|\Sigma_s|}$ reports the proportion of discovered CFDs that are valid on r, while $\text{recall}(\Sigma, \Sigma_s) = \frac{|\Sigma_s \cap \Sigma|}{|\Sigma|}$ measures the fraction of ground-truth CFDs recovered from r_s. Intuitively, precision quantifies the percentage of spurious rules generated by sampling, whereas recall quantifies the loss of CFDs incurred by the sampling process.

Given a CFD φ, and N independently and identically distributed samples $t_1, t_2, \ldots, t_N$ randomly drawn from the relation r, we define $X_i (1 \le i \le N)$ as an indicator variable where $X_i = 1$ if t_i conforms with φ and $X_i = 0$ otherwise. The sum $X = \sum_{i=1}^{N} X_i$ follows a binomial distribution $X \sim B(N, \hat{\sigma})$, with $E[X] = N\hat{\sigma}$ and

$Var(X) = N\hat{\sigma}(1-\hat{\sigma})$, where $\hat{\sigma} = \sigma/n$ is the normalized support. Let ϵ be the additive estimation error [6, 22] between the estimated and true normalized support, satisfying $Pr(|\hat{\sigma}_s - \hat{\sigma}| \leq \epsilon) \geq 1-\delta\beta$, where $\hat{\sigma}_s$ and $\hat{\sigma}$ denote the estimated and true normalized support, respectively. To derive a sampling lower bound N under recall guarantee β and error bound ϵ, we consider the following concentration inequalities. Notably, since X is a sum of Bernoulli trials, we can utilize the variance-sensitive form of concentration bounds (Bernstein-type) to obtain a tighter N compared to the standard Hoeffding's inequality [43]. First, we apply the variance-augmented Hoeffding-Bernstein inequality [11, 43]: $P_r(|X - E[X]| \geq N\epsilon) \leq 2\exp\left(\frac{-(N\epsilon)^2}{2Var(X)+\frac{2}{3}N\epsilon}\right) \approx 2\exp\left(\frac{-2(N\epsilon)^2}{Var(X)}\right) \leq 1-\delta\beta$. The use of $Var(X)$ is justified since it accounts for the distribution's sparsity; when $\hat{\sigma}$ is small (typical for CFD discovery), $Var(X) \ll N$, yielding a tighter bound than the non-parametric version. From this inequality, we can derive the variance-augmented lower bound: $N \geq \frac{\hat{\sigma}(1-\hat{\sigma})\ln(\frac{1-\delta\beta}{2})}{-2\epsilon^2}$.

We next compare the proposed variance-augmented lower bound with commonly used bounds derived from other sampling inequalities [6, 16, 22]. Specifically, using Chebyshev's inequality $P_r(|X - E[X]| \geq N\epsilon) \leq \frac{Var(X)}{(N\epsilon)^2} \leq 1-\delta\beta$ [22] , we obtain Chebyshev's lower bound $N \geq \frac{\hat{\sigma}(1-\hat{\sigma})}{\epsilon^2(1-\delta\beta)}$. According to Bennett's inequality $P_r(|X - N\hat{\sigma}| \geq N\epsilon) \leq 2\exp\left(-N\hat{\sigma}\Theta\left(\frac{\epsilon}{\hat{\sigma}}\right)\right) \leq 1-\delta\beta$ [6], we can derive Bennett's lower bound $N \geq \frac{-\ln(\frac{1-\delta\beta}{2})}{(\hat{\sigma}+\epsilon)\ln(1+\frac{\epsilon}{\hat{\sigma}})-\epsilon}$. Finally, a hybrid Chernoff bound was introduced in [16], given by $N \geq \frac{3\tau}{\epsilon^2}\ln\left(\frac{2}{1-\delta\beta}\right)$.

Theorem 1: *For $X \sim B(N, \hat{\sigma})$ where $0 \leq \hat{\sigma} \leq 1$, the variance-augmented lower bound is tighter than both Chebyshev's and Bennett's lower bounds.*

Proof. Please refer to Appendix C.1 of [1] for the proof. □

Theorem 2: *Given the normalized support threshold $\hat{\sigma}$, additive Chernoff error bound ϵ, error ratio τ, confidence δ, and recall β, when $X \sim B(N, \hat{\sigma})$ and $\hat{\sigma}(1-\hat{\sigma}) \leq 6\tau$, the sampling lower bound N is:*

$$N = \begin{cases} \frac{\hat{\sigma}(1-\hat{\sigma})\ln\left(\frac{2}{1-\delta\beta}\right)}{2\epsilon^2}, & \hat{\sigma} \in [0, \hat{\sigma}_1] \cup (\hat{\sigma}_2, 1] \text{ when } \tau \leq \frac{1}{24} \\ \frac{3\tau}{\epsilon^2}\ln\left(\frac{2}{1-\delta\beta}\right), & \hat{\sigma} \in (\hat{\sigma}_1, \hat{\sigma}_2] \text{ when } \tau \leq \frac{1}{24} \\ \frac{\hat{\sigma}(1-\hat{\sigma})\ln\left(\frac{2}{1-\delta\beta}\right)}{2\epsilon^2}, & \hat{\sigma} \in [0, 1] \text{ when } \tau > \frac{1}{24} \end{cases}$$

where $\hat{\sigma}_1, \hat{\sigma}_2 = \frac{1 \mp \sqrt{1-24\tau}}{2}$.

Proof. Please see Appendix C.2 of [1] for the proof. □

Example 5: *The following example illustrates how to derive the sample size bound in Theorem 2. Consider the parameters $\hat{\sigma} = 0.01$, $\epsilon = 0.001$, $\delta = 0.999$, $\tau = 0.02$, and $\beta = 1$.* AttrFinder *first computes the threshold $\hat{\sigma}_1 = \frac{1-\sqrt{1-24\times0.02}}{2} \approx 0.142$. Since the true normalized support satisfies $\hat{\sigma} = 0.01 \leq \hat{\sigma}_1$ for $\tau \leq \frac{1}{24}$, the required sample size is $N = \frac{0.01\times0.99\times\ln(2000)}{2\times0.001^2} \approx 37,624$. In contrast, the hybrid Chernoff bound [16] requires $N = \frac{3\times0.02}{0.001^2}\ln(2000) \approx 456,055$. Thus, the proposed bound reduces the required sample size by more than 91.7% while preserving the $(\epsilon, \delta\beta)$ guarantee.*

Remark. Compared to prior methods, our sampling strategy has two key characteristics: (1) Optimized bound tightness: incorporating binomial variance yields a tighter lower bound N than Hoeffding and hybrid Chernoff bounds, especially for low-support CFDs. (2) Dimensional scalability: N is independent of tuple count n, ensuring bounded overhead even on million-scale datasets.

7.2 Representative Tuple Sampling

Existing sampling methods often overlook low-support CFDs. To address this, we perform stratified sampling on the zero-one matrix M generated in Section 5.1, so that CFDs with low support are represented within smaller buckets. We then sample tuples from each bucket according to the sampling bound N derived in Section 7.1 to construct the set S. As a result, S satisfies the sampling guarantees while preserving the diversity of CFDs.

Similarity-based Representative Sampling. Since each row $m_i \in M$ is a concatenation of zero-one vectors, the inner product $m_i \cdot m_j$ counts the number of identical attribute values between tuples t_i and t_j. To ensure that both high- and low-support CFDs can be discovered, we propose similarity-aware stratified sampling. We employ locality sensitive hashing (LSH) [48] to group the rows of M into collision buckets based on the number of identical attribute values, facilitating representative sampling over the original relation r.

As shown in Algorithm 2, RepSampler samples representative tuples in the following manner to ensure the discovery of diverse CFDs. For the zero-one matrix M, RepSampler employs LSH based on the MinHash strategy [12] to map tuples into buckets $\mathcal{B} = \{B_1, \ldots, B_k\}$ (Line 4). Unlike density clustering [25], RepSampler converges efficiently by leveraging the MinHash strategy, which processes only the non-zero indices in each row, thereby avoiding unnecessary computation over zero entries. For each row m_i, RepSampler generates hash signatures in a single scan. The LSH mechanism ensures that tuples within the same bucket B_i share high Jaccard similarity [12] with high probability, indicating that they satisfy the same CFDs. The time complexity of this stage is $O(n)$ [48], where n is the number of tuples. LSH suits SCFDM well, compared to alternatives. Specifically, traditional random sampling lacks tuple similarities and tends to underrepresent rare tuples, which may omit low-support CFDs [73, 84]. Although frequency- or value-based stratified sampling mitigates distribution bias, constructing and maintaining strata over high-dimensional attribute combinations introduce substantial overhead [5, 58]. In contrast, LSH groups similar tuples through hash-based partitioning, which forms buckets that isolate and preserve rare tuple patterns with negligible computational cost.

RepSampler first removes any bucket B_i with $|B_i| < \sigma$, as such buckets cannot satisfy the support threshold of a CFD. It then processes the remaining buckets in ascending order of size, selecting one tuple from each non-empty bucket in each round. During this iterative process, buckets are only discarded once all their tuples have been selected, until the total sample size $|S|$ reaches the bound N (Lines 5–15). This strategy ensures that low-support CFD patterns in small buckets are preserved in the sample set S, preventing them from being ignored by high-support CFD patterns in large buckets. The number of samples selected across all buckets is constrained by the sampling lower bound N derived in Theorem 2 (Line 3). RepSampler preserves low-support CFDs by sampling from similarity-based buckets.

Algorithm 2 RepSampler (Similarity-based Stratified Sampling)

Input: Relation r, thresholds σ, δ, ϵ, Chernoff coefficient τ, LSH parameters L, zero-one encoding matrix M
Output: Representative sample set S
$N \leftarrow$ calculate sampling lower bound via Theorem 2
$\mathcal{B} = \{B_1, \ldots, B_k\} \leftarrow$ Partition M via LSH $h(\cdot)$ using L
$S \leftarrow \emptyset$, $\mathcal{B} \leftarrow$ sort buckets in $\mathcal{B}$ in ascending order of size $|B_i|$
while $|S| < N$ **and** $\mathcal{B}$ is nonempty **do**
  for each bucket $B_i \in \mathcal{B}$ **do**
    if B_i is nonempty **then**
      $s \leftarrow$ randomly sample a tuple from B_i
      $S \leftarrow S \cup \{s\}, \quad B_i \leftarrow B_i \setminus \{s\}$
      if $|S| = N$ **then**
        break
    else
      $\mathcal{B} \leftarrow \mathcal{B} \setminus \{B_i\}$
return S

Example 6: *We illustrate the stratified sampling workflow in Algorithm 2 using Table 1. Suppose LSH partitions the relation into buckets* $\mathcal{B} = \{B_1 = \{t_5\}, B_2 = \{t_3, t_4\}, B_3 = \{t_1, t_2, t_6, t_7\}\}$ *(Line 4), and the derived lower bound is* $N = 5$ *(Line 3). To prevent pruning of low-support patterns,* RepSampler *iteratively rotates through the buckets in* $\mathcal{B}$ *to sample tuples: (1) Iteration 1 (Lines 6–14): It randomly extracts* t_5 *from* B_1, t_3 *from* B_2, *and* t_2 *from* B_3, *yielding* $S = \{t_5, t_3, t_2\}$. *At this point,* B_1 *becomes empty and is pruned (Line 13). (2) Iteration 2: Since* $|S| = 3 < N$, *it continues sampling from the remaining non-empty buckets, randomly selecting* t_4 *from* B_2 *and* t_7 *from* B_3. *The sample size now reaches* $|S| = 5 = N$, *triggering the break condition (Line 11). The final representative set* $S = \{t_5, t_3, t_2, t_4, t_7\}$ *protects the rare bucket* B_1, *preserving the low-support pattern.*

Complexity. The time complexity of RepSampler is linear in the tuple count, i.e., $O(n)$, treating attributes and hash functions as constants (see Appendix G in [1] for details).

8 PARALLEL RULE DISCOVERY

We propose PCFDFinder, a CFD discovery framework that leverages correlated attribute sets (Section 6) and representative tuples (Section 7) to enable parallel execution of existing CFD mining algorithms. We first present its sub-table–based parallel design (Section 8.1), and then analyze its computational efficiency (Section 8.2).

8.1 Parallel Discovery Framework

To facilitate parallel discovery, we decompose the representative tuple set S (obtained in Section 7.2) into l sub-tables $\{r_{s1}, \ldots, r_{sl}\}$. This decomposition is guided by the correlated attribute sets $Corr = \{corr_1, \ldots, corr_l\}$ derived in Section 6.3, where each sub-table r_{si} is generated by projecting S onto the corresponding attribute set $corr_i$ (i.e., $r_{si} = \pi_{corr_i}(S)$). We then define cover $\Sigma_s = \bigcup \Sigma_{si}$, where each cover Σ_{si} is the set of minimal CFDs discovered from the corresponding sub-table r_{si}.

Next, we introduce PCFDFinder, a parallel framework for mining CFDs. As outlined in Algorithm 3, PCFDFinder aims to discover the minimal cover Σ_s of CFDs from r in parallel (line 2). PCFDFinder projects the representative tuple set S onto correlated attribute sets $Corr = \{corr_1, corr_2, \ldots, corr_l\}$, and obtains l input sub-tables $R_{sub} = \{r_{s1}, \ldots, r_{sl}\}$ (Line 3). Then, PCFDFinder spawns l threads

Algorithm 3 PCFDFinder (Data-parallel CFD mining)

Input: Correlated attribute sets $Corr = \{Corr_1, Corr_2, \ldots, Corr_l\}$, the representative tuple set S, a sequential CFD mining algorithm Λ, support σ, confidence δ, and the relation r
Output: A minimal CFD cover Σ_s satisfying thresholds
$R_{sub} = \{r_1, r_2, \ldots, r_l\} \leftarrow$ Generate from $Corr$ and S
Create l threads across multiple processors
for each thread t_i for $i \in [1, l]$ executing in parallel **do**
  $\Sigma_{si} \leftarrow$ Mine the cover of *minimal* CFDs from r_i using Λ
  for each CFD φ in Σ_{si} **do**
    Verify the CFD φ on r and remove rules from Σ_{si} that violate r
$\Sigma_s \leftarrow \Sigma_{s1} \cup \Sigma_{s2} \cup \ldots \cup \Sigma_{sl}$
return Σ_s

to concurrently run a sequential CFD discovery algorithm Λ on each sub-table r_{si} (Lines 4-6). Within each thread, every discovered candidate CFD φ is immediately validated against the original instance r with a complexity of $O(n)$, where n is the number of tuples in r, and CFDs violating r are discarded from local covers (Lines 7-8). Once all threads complete, PCFDFinder computes the union of the validated local covers: $\Sigma_s = \bigcup_{i=1}^{l} \Sigma_{si}$ (Line 9). The final cover Σ_s is returned as the cover of CFDs on relation r (Line 10).

8.2 The reduction of computation

The time complexity of the sequential CFD mining algorithm is $O(n \times m \times d^m)$ [72], where n, m and d represent the number of tuples, attributes, and the average domain size of attributes, respectively. With data parallelism in PCFDFinder, this time complexity is reduced to $O(n_i \times m_i \times {d_i}^{m_i})$, where n_i, m_i, and d_i are the number of tuples, attributes, and domain size in the largest sub-table. As a result, CFD discovery efficiency is improved by a factor of $\eta = O(\frac{n \times m \times d^m}{n_i \times m_i \times {d_i}^{m_i}}) \geq O(\frac{d^{(1-\kappa_2)\times m}}{\kappa_1 \times \kappa_2})$, where $d_i \leq d$, and κ_1, κ_2 denote the ratios of tuple counts and attribute counts in the largest sub-table relative to the original relation, respectively. In parallel execution, PCFDFinder delivers linear speedup via the tuple scaling factor κ_1^{-1} and exponential efficiency gains via the attribute reduction factor κ_2^{-1} across decomposed sub-tables.

9 EXPERIMENTAL STUDY

In this section, we experimentally evaluated (1) the efficiency of SCFDM; (2) the scalability of SCFDM; (3) the effectiveness (discovery accuracy) of SCFDM; (4) sensitivity to CFD core parameters; (5) the ablation study of SCFDM; and (6) a case study of CFDs on real-world and synthetic datasets.

9.1 Experimental Setting

Datasets. As shown in Table 2 (details in Appendix H.7 of [1]), we evaluate SCFDM on five real-world datasets (1–5) from UCI [3] and Kaggle [70], and two synthetic datasets (6–7) generated via Bayesian networks [78, 88, 91]: (1) RT-IOT (RT), covering IoT network traffic and cyberattacks; (2) Census-Income (Census), containing weighted 1994–1995 population survey data; (3) Crop, combining multi-modal remote sensing images; (4) Flight, recording domestic flight on-time performance; and (5) Adult, predicting census-based income levels. (6) Syn-1 and (7) Syn-2 provide controlled environments with verifiable ground-truth CFD covers and customizable noise for robustness evaluation. In Table 2, the "#Sampled"

Table 2: Summary of datasets and algorithm outputs

Dataset	Source	#Tuples	#Sampled	#Attributes	#CFDs	#Sub-tables
RT	UCI	123, 117	8, 519	85	50, 498	85
Census	UCI	199, 523	10, 544	45	84, 309	45
Crop	Kaggle	325, 835	16, 063	175	78, 232	175
Flight	Kaggle	1, 000, 000	41, 057	25	6, 224	25
Adult	UCI	47, 621	8, 422	14	435	14
Syn-1	bnlearn	20, 000	8, 408	48	2, 263	48
Syn-2	bnlearn	20, 000	8, 397	37	1, 755	37

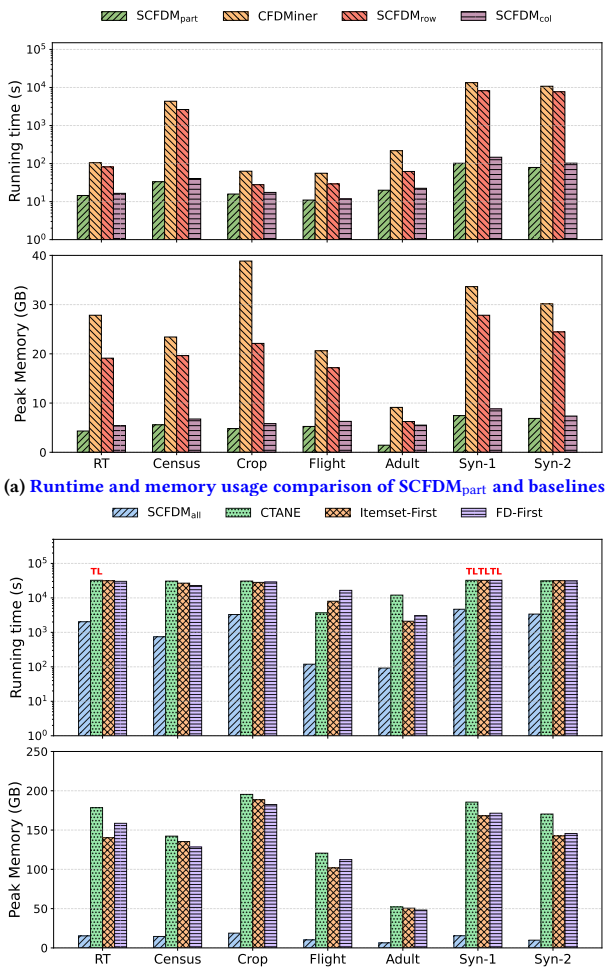
column reports the representative tuples selected by RepSampler, while the "#Sub-tables" column lists the partitioned sub-tables generated by SCFDM. Since AttrFinder extracts a correlated attribute set $Corr_Y$ for each attribute $Y \in attr(\mathcal{R})$, the number of sub-tables equals the number of attributes.

Algorithms. We implement eight CFD discovery algorithms in C++: (1) CFDMiner [29] for mining constant CFDs; (2) CTANE [29], a level-wise algorithm for mining both constant and variable CFDs; (3) Itemset-First [72] and (4) FD-First [72], two advanced hybrid CFD mining methods that alternate between itemset mining and FD discovery via depth-first or FD-first strategies; (5) $\text{SCFDM}_{\text{part}}$, our proposed constant CFD miner integrating AttrFinder, RepSampler, and PCFDFinder (with CFDMiner as the sequential engine Λ); (6) $\text{SCFDM}_{\text{all}}$, our proposed miner for both constant and variable CFDs, employing Itemset-First as Λ; and (7) $\text{SCFDM}_{\text{row}}$ and (8) $\text{SCFDM}_{\text{col}}$, two ablation variants of $\text{SCFDM}_{\text{part}}$ that omit AttrFinder and RepSampler, respectively. For fair comparison, we parallelized all four sequential baselines (CFDMiner, CTANE, Itemset-First, and FD-First) using parallel lattice traversal and multi-space search strategies [30, 52, 77] described in Section 2.

Ground truth of CFDs. For real-world datasets, we employ a dual-mining strategy within Metanome [65]: the level-wise CTANE [29] for exhaustive CFD discovery, supplemented by CFDMiner [29] for constant CFDs. For synthetic datasets, the ground truth Σ is inherently defined by the Bayesian generative process, where structural priors ensure that instances strictly adhere to the predefined dependency logic. Following automated generation/mining, manual expert review filters out redundant or invalid rules. This rigorous workflow yields a high-fidelity "gold standard" Σ for each dataset, with the total counts recorded in the "#CFDs" column of Table 2.

Accuracy metrics. Let Σ denote the ground-truth CFDs of the original relation r, and Σ_s the CFDs discovered from the partitioned sub-tables r_s. We evaluate SCFDM using the following metrics: (1) Precision (P) measures the proportion of discovered CFDs (Σ_s) that are valid on the original relation r: $P(\Sigma, \Sigma_s) = \frac{|\Sigma_s \cap \Sigma|}{|\Sigma_s|}$; (2) Recall ($R$) measures the fraction of ground-truth CFDs (Σ) successfully recovered in Σ_s: $R(\Sigma, \Sigma_s) = \frac{|\Sigma_s \cap \Sigma|}{|\Sigma|}$; (3) F1-score ($F_1$) is the harmonic mean of P and R: $F_1 = 2 \cdot \frac{P \cdot R}{P+R}$.

Default settings. Experiments run on a four-node cluster equipped with 224 GB RAM, where each node has two 28-core 2.70 GHz Intel Xeon CPUs. The Transformer training utilizes two NVIDIA GPUs (32 GB VRAM each) on one node. The embedding network uses a hidden dimension of 128 and is trained with a batch size of 128. optimized via AdamW with a learning rate of 5×10^{-4} for 20 epochs (patience = 3). Default parameters are: support $\sigma = 200$, confidence $\delta = 0.95$, support error bound $\epsilon = 5 \times 10^{-4}$, Chernoff ratio $\tau = 0.05$, and recall $\beta = 0.9$. Following [4, 42, 75], ϵ and τ are adaptively reduced for larger datasets (e.g., Crop and Flight) to better preserve low-support CFDs, which increases the sampled tuples in Table 2.



(a) Runtime and memory usage comparison of $\text{SCFDM}_{\text{part}}$ and baselines

(b) Runtime and memory usage comparison of $\text{SCFDM}_{\text{all}}$ and baselines

Figure 3: Efficiency evaluation of SCFDM and baselines

The time limits are 4 hours for constant CFD discovery and 9 hours for both constant and variable CFDs. We report the average of three runs. We provide a guidance of hyperparameter configuration in Appendix H.4 of [1].

9.2 Experimental Results

Exp-1: Efficiency of SCFDM. We evaluate the efficiency of SCFDM by comparing it with state-of-the-art baselines across all seven datasets. Figure 3 illustrates the execution time and peak memory usage, where "TL" denotes that a baseline exceeded the designated time limit (14, 400s for Figure 3(a) and 32, 400s for Figure 3(b)).

Figure 3(a) compares $\text{SCFDM}_{\text{part}}$, $\text{SCFDM}_{\text{row}}$, and $\text{SCFDM}_{\text{col}}$ against CFDMiner. $\text{SCFDM}_{\text{part}}$ consistently exhibits the highest efficiency, achieving an average speed-up of 105.18× across all datasets. Notably, on the Syn-2 dataset, CFDMiner took 10, 776.90s, whereas $\text{SCFDM}_{\text{part}}$ completed in 78.96s, yielding a speedup of 136.49×. The peak memory curves in Figure 3(a) demonstrate that the runtime improvements are accompanied by reduced memory consumption. While CFDMiner suffers from combinatorial lattice explosion, yielding an average peak memory consumption of 26.23GB (up to 38.85GB on Crop), $\text{SCFDM}_{\text{part}}$ reduces the average peak memory to 5.11GB, achieving a 5.13× reduction. Moreover, removing AttrFinder ($\text{SCFDM}_{\text{row}}$) increases average peak memory to

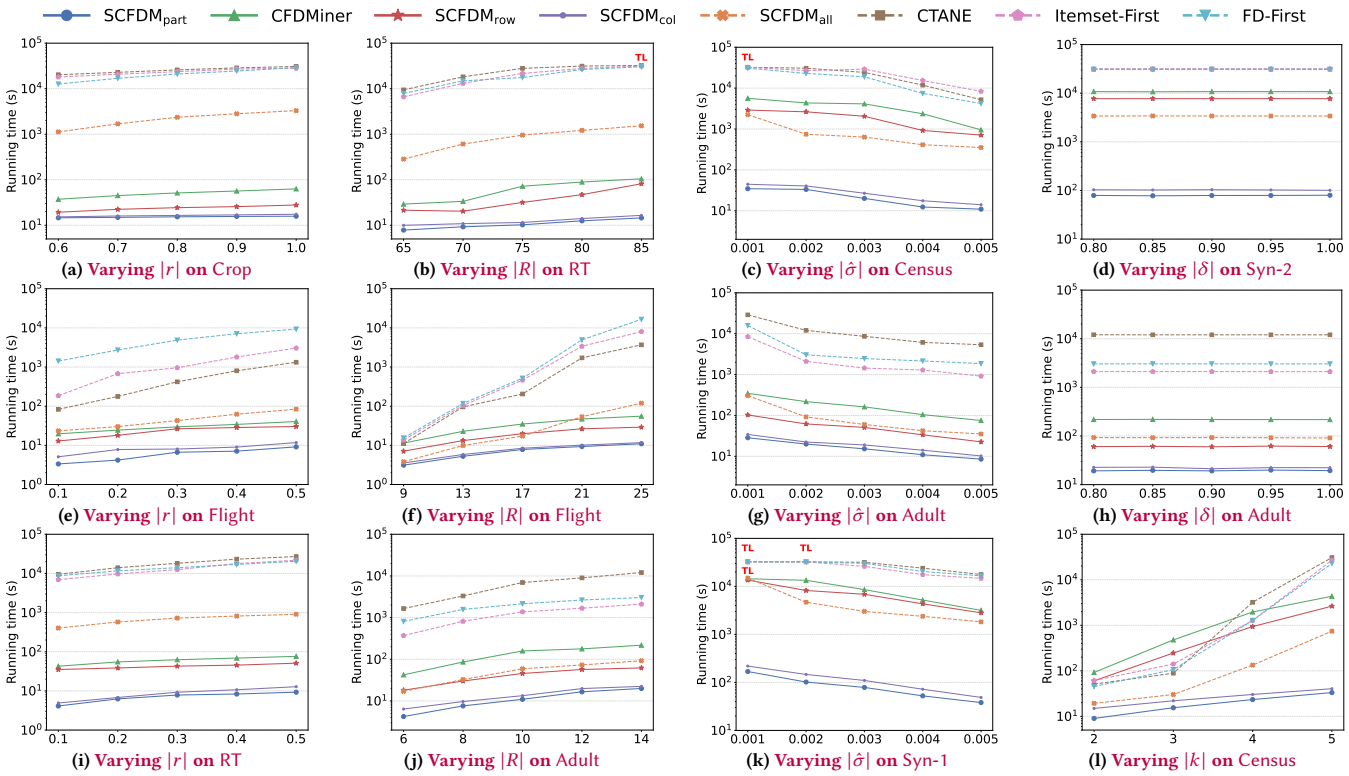


Figure 4: Scalability evaluation of SCFDM

19.51GB due to full attribute traversal, while omitting RepSampler (SCFDM$_{col}$) leads to an average rise to 6.56GB due to additional tuple processing. Together, these components effectively reduce memory consumption in CFD discovery.

Figure 3(b) evaluates SCFDM$_{all}$ against CTANE, Itemset-First, and FD-First. SCFDM$_{all}$ maintains a substantial lead, especially on complex datasets. On average, SCFDM$_{all}$ is at least 11.58× faster than three baselines for each dataset. Note that for datasets where baselines exceeded the time limit, we conservatively recorded the time as 32,400s (TL); the actual execution times are likely significantly higher. Specifically, it achieved a maximum speed-up of 130.78× on the Adult dataset compared to CTANE. On Syn-1, where all three baselines triggered the time limit, SCFDM$_{all}$ successfully completed the task in 4,682.95s, demonstrating its scalability in handling large search spaces. Figure 3(b) shows that SCFDM reduces memory consumption. Baselines suffer from severe memory pressure due to the expansion of lattices in the large search space, averaging peak memory footprints of 149.27GB (CTANE), 132.48GB (Itemset-First), and 135.33GB (FD-First) across all datasets, with CTANE peaking at 195.41GB on Crop. In contrast, SCFDM$_{all}$ cuts the average peak memory footprint to merely 12.96GB, delivering an 10.73× memory reduction on average. This validates that AttrFinder and RepSampler effectively reduce the memory footprint of constant and variable CFD discovery.

The reported runtime of SCFDM includes both data preprocessing and rule discovery, which includes the neural embedding training, correlation extraction, and data partitioning. Empirically, the overhead on embedding and Transformer training is trivial, requiring only 7.42–23.15s (13.17s on average). This overhead accounts for 29.90% of runtime in constant CFD discovery and just 0.55% in both constant and variable CFD discovery on average, dropping to 0.27% on large-scale datasets such as Crop. This shows that the embedding and Transformer training overhead becomes increasingly negligible as the search space grows. Detailed experimental results are provided in Appendix F of [1].

Analysis. There are two main reasons for the high efficiency of SCFDM: (1) partitioning relation r into sub-tables for parallel CFD discovery, which prunes the search space and reduces runtime and memory consumption; and (2) leveraging AttrFinder and RepSampler to identify correlated attributes and representative tuples, thereby reducing redundant attribute and tuple traversal. Moreover, the embedding and Transformer training overhead is modest. In contrast, baselines (e.g., CTANE) lack Transformer-based correlation extraction, resulting in a larger search space.

Exp-2: Scalability of SCFDM. We evaluated SCFDM's scalability by varying (1) the tuple sampling ratio $|r|$, (2) the number of attributes $|R|$, (3) the normalized support threshold $|\hat{\sigma}|$, (4) the confidence threshold $|\delta|$, (5) the maximum CFD size k, which limits the total number of involved attributes (LHS and RHS combined) in a single CFD, and (6) the number of CPU cores $|q|$.

Varying $|r|$. As shown in Figures 4(a), 4(e) and 4(i), the tuple sampling ratio $|r|$ was tuned from 0.6 to 1.0 on Crop and from 0.1 to 0.5 on Flight and RT. As $|r|$ grew, the running times of SCFDM$_{part}$, SCFDM$_{all}$, SCFDM$_{row}$, SCFDM$_{col}$ along with the baselines increased due to the expanded search space. Across the three datasets, SCFDM$_{part}$ achieved an average efficiency gain of 4.96× over CFDMiner. Meanwhile, SCFDM$_{all}$ outperformed the baselines (CTANE, Itemset-First, and FD-First) with an average speedup of 13.77×. Specifically, SCFDM$_{part}$ achieved a 3.32× efficiency gain over CFDMiner on Crop, while SCFDM$_{all}$ exhibited higher improvements, outperforming the baselines by 47.88× and 22.62×

Table 3: Parallel performance of SCFDM with varying $|q|$

Algorithm	Dataset	Execution Time (s)			
		$\|q\| = 28$	$\|q\| = 56$	$\|q\| = 84$	$\|q\| = 112$
$SCFDM_{part}$	RT	35.41	22.23	17.80	14.48
$SCFDM_{all}$	RT	4,904.92	3,422.11	2,654.12	2,031.53
$SCFDM_{part}$	Census	75.33	52.13	41.03	33.27
$SCFDM_{all}$	Census	1,689.25	1,274.70	908.41	744.68
$SCFDM_{part}$	Adult	45.35	32.61	26.62	19.93
$SCFDM_{all}$	Adult	217.08	133.11	102.66	92.18
$SCFDM_{part}$	Syn-1	188.32	159.78	117.84	101.85
$SCFDM_{all}$	Syn-1	12,240.28	8,201.08	6,102.75	4,682.95

on Flight and RT on average, respectively.

Varying $|R|$. Figures 4(b), 4(f), 4(j) show the impact of varying attribute number $|R|$ across three datasets: RT (65–85), Flight (9–25), and Adult (6–14). As $|R|$ increased, the runtimes of all baselines escalated significantly. In contrast, $SCFDM_{part}$, $SCFDM_{row}$, $SCFDM_{col}$, and $SCFDM_{all}$ exhibited much more resilient and gradual growth. Specifically, $SCFDM_{part}$ outperformed CFDMiner by 11.51× on Adult, while $SCFDM_{all}$ achieved higher speedups, which beat the baselines by an average of 23.00×, 65.03× and 60.24× on RT, Flight and Adult, respectively. Figure 4(b) highlights that $SCFDM_{col}$ outperforms $SCFDM_{row}$ in speedup.

Varying $|\hat{\sigma}|$. Figures 4(c), 4(g), 4(k) illustrate the impact of varying the normalized support threshold $|\hat{\sigma}|$ (as defined in Section 7.1) from 1×10^{-3} to 5×10^{-3} across Census, Adult and Syn-1. As $|\hat{\sigma}|$ increased, all baselines saw a significant decrease in execution time due to more candidate pruning. Specifically, $SCFDM_{part}$ and $SCFDM_{all}$ consistently outperformed the baselines, achieving average speedups of 156.58× (resp. 22.84×) on Census, 10.91× (resp. 62.82×) on Adult, and 101.70× (resp. 4.90×) on Syn-1.

Varying $|\delta|$. In Figures 4(d) and 4(h), we found that the execution times of SCFDM, CTANE, Itemset-First, and FD-First showed little change when varying the confidence threshold $|\delta|$ from 0.80 to 1.00. This is because $|\delta|$ has a limited effect on the search space. Specifically, $SCFDM_{part}$ and $SCFDM_{all}$ achieved average speedups of 136.49× (resp. 9.19×) on Syn-2 and 11.20× (resp. 62.16×) on Adult.

Varying $|k|$. We varied the maximum CFD size $|k|$ from 2 to 5 on Census. Figure 4(l) shows that as $|k|$ increased, the running times of SCFDM, CTANE, Itemset-First, and FD-First grew exponentially, with SCFDM growing more slowly. On average, $SCFDM_{part}$ and $SCFDM_{all}$ were 84.77× and 31.14× more efficient, respectively. See Appendix H.3 in [1] for the setting guidance of k.

Varying $|q|$. We increased the number of CPU cores $|q|$ from 28 to 112 to assess the scalability of both $SCFDM_{part}$ and $SCFDM_{all}$ on the RT, Census, Adult, and Syn-1 datasets. Table 3 shows that SCFDM scales effectively with more CPU cores. As $|q|$ increased, the average runtime of SCFDM across the four datasets decreased from 2,424.5s to 965.1s. Specifically, $SCFDM_{part}$ and $SCFDM_{all}$ achieved an average speedup of 2.03× and 2.52× across the four datasets, respectively.

Analysis. The experimental results demonstrate the effectiveness of SCFDM. (1) Its execution time varies most with $|R|$ and $|k|$, due to their exponential effect on the search space of sub-tables. (2) SCFDM scales well with parameters $|r|$ and $|\hat{\sigma}|$ by sampling a fixed number of representative tuples and efficiently discovering CFDs on sub-tables. (3) The parameter $|\delta|$ has little impact on the search space. (4) Additionally, increasing the number of CPU cores $|q|$ speeds up SCFDM by distributing the workload across CPU cores with low communication overhead.

Table 4: Effectiveness of CFD discovery

Dataset	Metric	CFDMiner	CTANE	Itemset-First	FD-First	SCFDM
RT	Precision	0.985	0.984*	0.974	0.972	**0.986**
	Recall	0.075	0.709*	**0.966**	0.962	0.960
	F_1	0.139	0.824*	0.970	0.967	**0.973**
Census	Precision	0.982	0.952	0.958	0.966	**0.987**
	Recall	0.062	0.990	0.988	**0.991**	0.983
	F_1	0.117	0.971	0.973	0.978	**0.985**
Crop	Precision	0.970	0.952	0.946	0.950	**0.971**
	Recall	0.084	0.949	**0.952**	0.949	0.933
	F_1	0.155	0.950	0.949	0.949	**0.952**
Flight	Precision	**0.999**	0.996	0.995	0.995	0.997
	Recall	0.140	0.998	**0.999**	**0.999**	0.998
	F_1	0.246	**0.997**	**0.997**	**0.997**	**0.997**
Adult	Precision	**1.000**	**1.000**	**1.000**	**1.000**	**1.000**
	Recall	0.338	**1.000**	**1.000**	**1.000**	**1.000**
	F_1	0.505	**1.000**	**1.000**	**1.000**	**1.000**
Syn-1	Precision	0.977	**0.979***	0.960*	0.966*	**0.979**
	Recall	0.041	0.833*	0.872*	0.868*	**0.953**
	F_1	0.079	0.900*	0.914*	0.914*	**0.966**
Syn-2	Precision	**0.985**	0.979	0.964	0.960	0.980
	Recall	0.079	0.962	**0.970**	**0.970**	0.966
	F_1	0.146	0.970	0.967	0.965	**0.973**

Results obtained before reaching the time (9 hours)/memory (224GB) limit.
* indicates that the algorithm timed out before completion. **Bold** indicates the best performance.

Exp-3: Effectiveness of SCFDM. We evaluated the effectiveness of SCFDM on all seven datasets. As shown in Table 4, SCFDM consistently achieves the highest F_1-score across all datasets. While all methods generally maintain high precision, they exhibit different performances on recall. Specifically, CFDMiner shows the lowest recall (e.g., only 0.062 on Census). Although it operates with relatively high efficiency, it is limited to discovering constant CFDs, thus missing a large number of variable CFDs. As for CTANE, Itemset-First, and FD-First, the low F_1-scores stem from (1) a large search space, which triggers the 9-hour timeout limit on large-scale datasets (e.g., Syn-1). and (2) the absence of attribute-correlation analysis, which introduces spurious CFDs and consequently degrades precision. In contrast, SCFDM strikes a balance by reducing the search space through attribute-aware pruning and representative sampling, achieving high accuracy while maintaining strong scalability. Specifically, on the RT and Syn-1 datasets, SCFDM achieves highest F_1-scores of 0.973 and 0.966, respectively, surpassing all baselines. To ensure a fair evaluation for CFDMiner, we provide a comparison restricted to constant CFD discovery in Appendix H.1 in [1].

Exp-4: Sensitivity to CFD Parameters. We evaluated the impact of CFD parameters (support thresholds $|\hat{\sigma}|$) on the RT and Syn-1 datasets based on the F_1-score. As shown in Table 5, SCFDM demonstrates robustness to variations in support thresholds. Specifically, when the threshold is small (e.g., $|\hat{\sigma}| = 1 \times 10^{-3}$), the F_1-scores of all baselines drop sharply, with certain methods falling below 0.5 (e.g., 0.082 for CFDMiner and 0.488 for CTANE on RT). This is because lower support thresholds expand the search space, causing baselines to trigger timeouts and fail to discover the full set of CFDs. In contrast, SCFDM consistently maintains high F_1-scores (above 0.90) even at $|\hat{\sigma}| = 1 \times 10^{-3}$. As $|\hat{\sigma}|$ increases to 5×10^{-3}, the performance gap narrows since the search space becomes more manageable for baselines. However, SCFDM still yields the best or

Table 5: Impact of normalized support thresholds $|\hat{\sigma}|$ on F_1-score across different algorithms

Dataset	Algorithm	Metric	1×10^{-3}	2×10^{-3}	3×10^{-3}	4×10^{-3}	5×10^{-3}
RT	CFDMiner	F_1	0.082	0.115	0.148	0.172	0.184
	CTANE		0.488	0.824	0.965	0.970	0.972
	Itemset-First		0.523	**0.970**	0.962	0.969	0.976
	FD-First		0.534	0.967	0.962	0.972	**0.982**
	SCFDM		**0.901**	**0.970**	**0.974**	**0.978**	**0.982**
Syn-1	CFDMiner	F_1	0.076	0.079	0.095	0.114	0.119
	CTANE		0.372	0.900	0.926	0.962	**0.977**
	Itemset-First		0.454	0.914	0.963	**0.971**	0.972
	FD-First		0.423	0.914	0.965	0.968	0.970
	SCFDM		**0.927**	**0.952**	**0.966**	**0.971**	**0.977**

Results obtained before reaching the time (9 hours)/memory (224GB) limit.

competitive results across all cases. These results demonstrate the robustness of SCFDM under low support thresholds, consistently achieving high discovery accuracy. Due to space limitations, we present the parameter sensitivity results on the other five datasets in Appendix H.2 of [1].

Exp-5: Ablation Study of SCFDM. To ensure fair runtime comparison, we conduct ablation studies on a random sample of 80,000 rows from Crop, ensuring that all variants complete execution within the time limit. As shown in Table 6, all three components are essential for balancing quality and efficiency. Omitted RepSampler elevates the runtime to 3,680.8s. Notably, omitting AttrFinder or RepSampler slightly increases recall (to 99.0% and 98.4%) due to exhaustive exploration over the complete relation, however, it severely degrades precision by introducing semantic noise and spurious CFDs (to 96.5% and 97.8%). Furthermore, PCFDFinder is critical for efficiency; its removal spikes runtime to 18,621.3s.

Evaluating alternative components highlights the effectiveness of SCFDM. First, replacing AttrFinder with Pearson correlation coefficient [34] (computing pairwise attribute correlation), XGBoost [14] or Kamino [38] compromises effectiveness as they fail to extract determinant antecedent sets for target attributes. For instance, XGBoost produces redundant attribute combinations, reducing its F_1-score to 79.8% (with recall dropping to 68.2%) and causing a runtime of 11,025.1s. Second, substituting RepSampler with random [63] or stratified [61] sampling yields only a limited reduction in sampling time (down to 642.2s) but at a heavy degradation in discovery quality, where the F_1-scores of the two methods drop from 98.2% to 86.4% and 88.2%, respectively, due to severe recall loss. This underscores that alternative sampling strategies cannot sufficiently preserve the tuples essential for CFD discovery.

Overall, these results demonstrate that the combined effects of AttrFinder, RepSampler, and PCFDFinder is essential for SCFDM to achieve both accuracy and runtime scalability.

Exp-6: Case study on data repair. We conducted a case study on the noisy Adult dataset to evaluate SCFDM's effectiveness in improving data quality. For example, some tuples with "Relationship = Husband" are mislabeled as "Female" in "Gender", and some cells contain missing values (marked as "?"). SCFDM identified important rules like φ_5 : ([Education, Occupation] $\rightarrow$ Workclass, (some-college, _ | _)) and φ_6 : ([Gender, Hours, Occupation] $\rightarrow$ Marital, (male, 50, sales | married-civ-spouse)). φ_5 suggests that a "some-college" education may limit workclass options; for instance, federal government jobs often require higher degrees. We use φ_5 to correct tuples with inconsistent workclasses and educational backgrounds. Meanwhile, φ_6 indicates that men around 50 working over 50 hours per week in sales are likely married and supporting a family. We apply φ_6 to correct tuples where such individuals are not listed as married. In contrast, baselines suffer from semantic noise and lattice explosions, producing spurious and redundant CFDs while failing to discover rules such as φ_5 and φ_6 that involve correlated attributes. Without attribute correlation analysis, the search space becomes dominated by spurious and redundant CFDs, preventing the discovery of CFDs for data cleaning in large datasets.

Table 6: Ablation study of key SCFDM components on Crop

Model Configuration	Prec. (%)	Rec. (%)	F1 (%)	Runtime (s)
SCFDM (Full Model)	**98.6**	97.9	**98.2**	655.4
Ablations				
w/o AttrFinder	96.5	**99.0**	97.7	28,435.7
w/o RepSampler	97.8	98.4	98.1	3,680.8
w/o PCFDFinder	**98.6**	97.9	**98.2**	18,621.3
Alternative Components				
w/o AttrFinder + w/ Pearson [34]	97.0	57.4	72.1	5,923.1
w/o AttrFinder + w/ XGBoost [14]	96.1	68.2	79.8	11,025.1
w/o AttrFinder + w/ Kamino [38]	98.0	76.5	85.9	4,388.2
w/o RepSampler + w/ random [63]	97.5	77.6	86.4	**642.2**
w/o RepSampler + w/ stratified [61]	98.2	80.1	88.2	644.6

Summary. We find the following: (1) Efficiency. SCFDM significantly outperforms all baselines. It achieves an average runtime speedup of 105.18× in constant CFD discovery (up to 136.49× on Syn-2) and at least 11.58× in both constant and variable CFD discovery. Furthermore, SCFDM reduces average peak memory to 5.11GB (5.13× reduction) for constant CFD discovery and 12.96GB (10.73× reduction) for both constant and variable CFD discovery, ensuring high memory efficiency. (2) Robust scalability. SCFDM scales robustly across all parameters, maintaining gradual runtime growth where baselines grow exponentially with increasing $|R|$ or $|k|$. (3) High discovery quality. SCFDM consistently achieves the highest F_1-scores (above 0.90), balancing precision and recall better than all baselines. (4) Component vitality. Ablation confirms that AttrFinder, RepSampler, and PCFDFinder are indispensable, as they together outperform alternatives (Pearson correlation, XGBoost, Kamino, Random sampling, and Stratified sampling) by filtering semantic noise and spurious candidate CFDs. (5) Efficient parallelism. SCFDM scales efficiently with computational resources, reducing average runtime from 2,424.5s to 965.1s and achieving a 2.03× to 2.52× speedup when the number of CPU cores $|q|$ increases from 28 to 112. (6) Practical utility. The case study shows that SCFDM discovers high-quality CFDs for repairing noisy data, whereas baselines produce spurious CFDs and fail to identify valid constraints (e.g., φ_5 and φ_6) within the time limits.

10 CONCLUSION

We propose SCFDM, a data-parallel framework for efficient CFD discovery on large-scale datasets. Its key innovations include: (1) Probabilistic representation: Modeling CFDs as conditional probabilities and learning attribute correlations with a two-stage Transformer. (2) Attention-driven extraction: AttrFinder derives correlated attribute sets from global attention to partition the relation into sub-tables. (3) Diversity-aware sampling: RepSampler reduces the input size while preserving discovery recall through a theoretically grounded sampling bound. (4) Data parallelism: Independent sub-tables are mined in parallel for scalable CFD discovery.

Our future work will focus on extending SCFDM to support more complex rules with machine learning predicates (e.g., REEs [32]).

REFERENCES

[1] 2026. Code, datasets and full version. https://anonymous.4open.science/r/CFDs-2026-EE65/.

[2] Ziawasch Abedjan, Lukasz Golab, and Felix Naumann. 2015. Profiling relational data a survey. *The VLDB Journal* 24 (2015), 557–581.

[3] Arthur Asuncion, David Newman, et al. 2007. UCI machine learning repository.

[4] Tuğkan Batu, Lance Fortnow, Ronitt Rubinfeld, Warren D Smith, and Patrick White. 2013. Testing closeness of discrete distributions. *Journal of the ACM (JACM)* 60, 1 (2013), 1–25.

[5] David R Bellhouse. 2005. Systematic sampling methods. *Encyclopedia of Biostatistics* 8 (2005).

[6] George Bennett. 1962. Probability inequalities for the sum of independent random variables. *J. Amer. Statist. Assoc.* 57, 297 (1962), 33–45.

[7] H Russell Bernard, Amber Wutich, and Gery W Ryan. 2016. *Analyzing qualitative data: Systematic approaches*. SAGE publications.

[8] George Beskales, Ihab F Ilyas, Lukasz Golab, and Artur Galiullin. 2014. Sampling from repairs of conditional functional dependency violations. *The VLDB Journal* 23 (2014), 103–128.

[9] Tobias Bleifuß, Susanne Bülow, Johannes Frohnhofen, Julian Risch, Georg Wiese, Sebastian Kruse, Thorsten Papenbrock, and Felix Naumann. 2016. Approximate discovery of functional dependencies for large datasets. In *Proceedings of the 25th ACM International on Conference on Information and Knowledge Management*. 1803–1812.

[10] Tobias Bleifuß, Sebastian Kruse, and Felix Naumann. 2017. Efficient denial constraint discovery with hydra. *Proceedings of the VLDB Endowment* 11, 3 (2017), 311–323.

[11] Stéphane Boucheron, Gábor Lugosi, and Pascal Massart. 2013. *Concentration Inequalities: A Nonasymptotic Theory of Independence*. Oxford University Press. https://doi.org/10.1093/acprof:oso/9780199535255.001.0001

[12] Andrei Z Broder. 1997. On the resemblance and containment of documents. In *Proceedings. Compression and Complexity of SEQUENCES 1997 (Cat. No. 97TB100171)*. IEEE, 21–29.

[13] Venkatesan T Chakaravarthy, Vinayaka Pandit, and Yogish Sabharwal. 2009. Analysis of sampling techniques for association rule mining. In *Proceedings of the 12th international conference on database theory*. 276–283.

[14] Tianqi Chen and Carlos Guestrin. 2016. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*. 785–794.

[15] Fei Chiang and Renée J Miller. 2008. Discovering data quality rules. *Proceedings of the VLDB Endowment* 1, 1 (2008), 1166–1177.

[16] Fan Chung and Linyuan Lu. 2002. Connected components in random graphs with given expected degree sequences. *Annals of combinatorics* 6, 2 (2002), 125–145.

[17] Zhang Chunsheng and Diao Yufeng. 2015. Conditional functional dependency discovery and data repair based on decision tree. In *2015 12th International Conference on Fuzzy Systems and Knowledge Discovery (FSKD)*. IEEE, 864–868.

[18] Edgar F Codd et al. 1972. *Relational completeness of data base sublanguages*. Citeseer.

[19] Graham Cormode, Lukasz Golab, Korn Flip, Andrew McGregor, Divesh Srivastava, and Xi Zhang. 2009. Estimating the confidence of conditional functional dependencies. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of data*. 469–482.

[20] Gonçalo M Correia, Vlad Niculae, and André FT Martins. 2019. Adaptively sparse transformers. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 2174–2184.

[21] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 4171–4186.

[22] Jay L Devore. 2000. Probability and statistics. *Pacific Grove: Brooks/Cole* (2000).

[23] Thierno Diallo, Nöel Novelli, and Jean-Marc Petit. 2012. Discovering (frequent) constant conditional functional dependencies. *International Journal of Data Mining, Modelling and Management* 4, 3 (2012), 205–223.

[24] Yuefeng Du, Derong Shen, Tiezheng Nie, Yue Kou, and Ge Yu. 2017. Discovering context-aware conditional functional dependencies. *Frontiers of Computer Science* 11 (2017), 688–701.

[25] Martin Ester, Hans-Peter Kriegel, Jörg Sander, Xiaowei Xu, et al. 1996. A density-based algorithm for discovering clusters in large spatial databases with noise. In *kdd*, Vol. 96. 226–231.

[26] Wenfei Fan. 2008. Dependencies revisited for improving data quality. In *Proceedings of the twenty-seventh ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. 159–170.

[27] Wenfei Fan. 2015. Data quality From theory to practice. *Acm Sigmod Record* 44, 3 (2015), 7–18.

[28] Wenfei Fan, Floris Geerts, Xibei Jia, and Anastasios Kementsietsidis. 2008. Conditional functional dependencies for capturing data inconsistencies. *TODS* (2008).

[29] Wenfei Fan, Floris Geerts, Jianzhong Li, and Ming Xiong. 2011. Discovering conditional functional dependencies. *TKDE* 23, 5 (2011), 683–698.

[30] Wenfei Fan, Ziyan Han, Yaoshu Wang, and Min Xie. 2022. Parallel Rule Discovery from Large Datasets by Sampling. In *Proceedings of the 2022 International Conference on Management of Data*. 384–398.

[31] Wenfei Fan, Ziyan Han, Yaoshu Wang, and Min Xie. 2023. Discovering Top-k Rules using Subjective and Objective Criteria. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–29.

[32] Wenfei Fan, Chao Tian, Yanghao Wang, and Qiang Yin. 2021. Parallel discrepancy detection and incremental detection. *Proceedings of the VLDB Endowment* 14, 8 (2021), 1351–1364.

[33] Ronald A Fisher. 1949. The design of experiments. (1949).

[34] Ronald Aylmer Fisher. 1970. Statistical methods for research workers. In *Breakthroughs in statistics: Methodology and distribution*. Springer, 66–70.

[35] Hector Garcia-Molina, Jeffrey D Ullman, and Jennifer Widom. 2000. *Database system implementation*. Vol. 672. Prentice Hall Upper Saddle River.

[36] Eve Garnaud, Nicolas Hanusse, Sofian Maabout, and Noël Novelli. 2014. Parallel mining of dependencies. In *2014 International Conference on High Performance Computing & Simulation (HPCS)*. IEEE, 491–498.

[37] Walter Gautschi. 2011. *Numerical analysis*. Springer Science & Business Media.

[38] Chang Ge, Shubhankar Mohapatra, Xi He, and Ihab F. Ilyas. 2021. Kamino: constraint-aware differentially private data synthesis. *Proc. VLDB Endow.* 14, 10 (2021), 1886–1899.

[39] Bart Goethals, Wim Le Page, and Heikki Mannila. 2008. Mining association rules of simple conjunctive queries. In *Proceedings of the 2008 SIAM International Conference on Data Mining*. SIAM, 96–107.

[40] Lukasz Golab, Howard Karloff, Flip Korn, Divesh Srivastava, and Bei Yu. 2008. On generating near-optimal tableaux for conditional functional dependencies. *Proceedings of the VLDB Endowment* 1, 1 (2008), 376–390.

[41] Yury Gorishniy, Ivan Rubachev, Valentin Khrulkov, and Artem Babenko. 2021. Revisiting deep learning models for tabular data. *Advances in neural information processing systems* 34 (2021), 18932–18943.

[42] Peter J Haas, Jeffrey F Naughton, S Seshadri, and Lynne Stokes. 1995. Sampling-based estimation of the number of distinct values of an attribute. In *VLDB*, Vol. 95. 311–322.

[43] Wassily Hoeffding. 1994. Probability inequalities for sums of bounded random variables. *The collected works of Wassily Hoeffding* (1994), 409–426.

[44] Xin Huang, Ashish Khetan, Milan Cvitkovic, and Zohar Karnin. 2020. Tabtransformer: Tabular data modeling using contextual embeddings. *arXiv preprint arXiv:2012.06678* (2020).

[45] Xin Huang, Ashish Khetan, Milan Cvitkovic, and Zohar Karnin. 2020. TabTransformer: Tabular Data Modeling Using Contextual Embeddings. (12 2020). https://doi.org/10.48550/arXiv.2012.06678

[46] Ykä Huhtala, Juha Kärkkäinen, Pasi Porkka, and Hannu Toivonen. 1999. TANE: An efficient algorithm for discovering functional and approximate dependencies. *The computer journal* (1999).

[47] Saxena H Golab L Ilyas IF. 2019. Distributed implementations of dependency discovery algorithms. *Proc. VLDB Endow* 12, 11 (2019), 1624.

[48] Piotr Indyk and Rajeev Motwani. 1998. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the thirtieth annual ACM symposium on Theory of computing*. 604–613.

[49] Sijia Jiang, Zijing Tan, Jiawei Wang, Zhikang Wang, and Shuai Ma. 2023. Guided conditional functional dependency discovery. *Information Systems* 114 (2023), 102158.

[50] Jyrki Kivinen and Heikki Mannila. 1995. Approximate inference of functional dependencies from relations. *Theoretical Computer Science* 149, 1 (1995), 129–149.

[51] Jan Kossmann, Thorsten Papenbrock, and Felix Naumann. 2022. Data dependencies for query optimization a survey. *The VLDB Journal* 31, 1 (2022), 1–22.

[52] Sebastian Kruse and Felix Naumann. 2018. Efficient discovery of approximate dependencies. *Proceedings of the VLDB Endowment* 11, 7 (2018), 759–772.

[53] Charles E Leiserson and Tao B Schardl. 2010. A work-efficient parallel breadth-first search algorithm (or how to cope with the nondeterminism of reducers). In *Proceedings of the twenty-second annual ACM symposium on Parallelism in algorithms and architectures*. 303–314.

[54] Jiuyong Li, Jixue Liu, Hannu Toivonen, and Jianming Yong. 2013. Effective pruning for the discovery of conditional functional dependencies. *Comput. J.* 56, 3 (2013), 378–392.

[55] Mingda Li, Hongzhi Wang, and Jianzhong Li. 2019. Mining conditional functional dependency rules on big data. *Big Data Mining and Analytics* 3, 1 (2019), 68–84.

[56] Yanrong Li and Raj P Gopalan. 2005. Stratified Sampling for Association Rules Mining. In *Artificial Intelligence Applications and Innovations: IFIP TC12 WG12. 5-Second IFIP Conference on Artificial Intelligence Applications and Innovations (AIAI2005), September 7–9, 2005, Beijing, China 2*. Springer, 79–88.

[57] Ester Livshits, Alireza Heidari, Ihab F Ilyas, and Benny Kimelfeld. 2020. Approximate Denial Constraints. *arXiv preprint arXiv:2005.08540* (2020).

[58] Sharon L Lohr. 2021. *Sampling: design and analysis*. Chapman and Hall/CRC.

[59] Ilya Loshchilov and Frank Hutter. 2017. Fixing Weight Decay Regularization in Adam. *CoRR* abs/1711.05101 (2017). arXiv:1711.05101 http://arxiv.org/abs/1711.05101

[60] James B McQueen. 1967. Some methods of classification and analysis of multivariate observations. In *Proc. of 5th Berkeley Symposium on Math. Stat. and Prob.* 281–297.

[61] Jerzy Neyman. 1992. On the two different aspects of the representative method: the method of stratified sampling and the method of purposive selection. In *Breakthroughs in statistics: Methodology and distribution*. Springer, 123–150.

[62] Frank Olken. 1993. *Random sampling from databases*. Ph.D. Dissertation. Citeseer.

[63] Frank Olken and Doron Rotem. 1986. Simple random sampling from relational databases. (1986).

[64] Kamlesh Kumar Pandey and Diwakar Shukla. 2020. Stratified sampling-based data reduction and categorization model for big data mining. In *Communication and Intelligent Systems: Proceedings of ICCIS 2019*. Springer, 107–122.

[65] Thorsten Papenbrock, Tanja Bergmann, Moritz Finke, Jakob Zwiener, and Felix Naumann. 2015. Data profiling with metanome. *Proceedings of the VLDB Endowment* 8, 12 (2015), 1860–1863.

[66] Thorsten Papenbrock, Jens Ehrlich, Jannik Marten, Tommy Neubert, Jan-Peer Rudolph, Martin Schönberg, Jakob Zwiener, and Felix Naumann. 2015. Functional dependency discovery: An experimental evaluation of seven algorithms. *PVLDB* 8, 10 (2015), 1082–1093.

[67] Thorsten Papenbrock and Felix Naumann. 2016. A hybrid approach to functional dependency discovery. In *Proceedings of the 2016 International Conference on Management of Data*. 821–833.

[68] Thorsten Papenbrock and Felix Naumann. 2017. Data-driven Schema Normalization.. In *EDBT*, Vol. 17. 342–353.

[69] Eduardo HM Pena, Fabio Porto, and Felix Naumann. 2022. Fast Algorithms for Denial Constraint Discovery. *Proceedings of the VLDB Endowment* 16, 4 (2022), 684–696.

[70] Luigi Quaranta, Fabio Calefato, and Filippo Lanubile. 2021. Kgtorrent: A dataset of python jupyter notebooks from kaggle. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. IEEE, 550–554.

[71] Joeri Rammelaere and Floris Geerts. 2018. Explaining repaired data with CFDs. *Proceedings of the VLDB Endowment* 11, 11 (2018), 1387–1399.

[72] Joeri Rammelaere and Floris Geerts. 2019. Revisiting conditional functional dependency discovery: Splitting the "C" from the "FD". In *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2018, Dublin, Ireland, September 10–14, 2018, Proceedings, Part II 18*. Springer, 552–568.

[73] Matteo Riondato and Eli Upfal. 2014. Efficient discovery of association rules and frequent itemsets through sampling with tight performance guarantees. *ACM Transactions on Knowledge Discovery from Data (TKDD)* 8, 4 (2014), 1–32.

[74] Sheldon M Ross, Sheldon M Ross, Sheldon M Ross, Sheldon M Ross, and Etats-Unis Mathématicien. 1998. *A first course in probability*. Vol. 6. Prentice Hall Upper Saddle River, NJ.

[75] Diego Santoro, Andrea Tonon, and Fabio Vandin. 2020. Mining sequential patterns with VC-dimension and rademacher complexity. *Algorithms* 13, 5 (2020), 123.

[76] Carl-Erik Särndal, Bengt Swensson, and Jan Wretman. 2003. *Model assisted survey sampling*. Springer Science & Business Media.

[77] Hemant Saxena, Lukasz Golab, and Ihab F Ilyas. 2019. Distributed discovery of functional dependencies. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*. IEEE, 1590–1593.

[78] Marco Scutari. 2010. Learning Bayesian networks with the bnlearn R package. *Journal of statistical software* 35 (2010), 1–22.

[79] Claude Elwood Shannon. 1948. A mathematical theory of communication. *The Bell system technical journal* 27, 3 (1948), 379–423.

[80] BS Sharmila and Rohini Nagapadma. 2023. Quantized autoencoder (QAE) intrusion detection system for anomaly detection in resource-constrained IoT devices using RT-IoT2022 dataset. *Cybersecurity* 6, 1 (2023), 41.

[81] Shaoxu Song and Lei Chen. 2009. Discovering matching dependencies. In *Proceedings of the 18th ACM Conference on Information and Knowledge Management, CIKM 2009, Hong Kong, China, November 2-6, 2009*. ACM, 1421–1424.

[82] Shaoxu Song and Lei Chen. 2011. Differential dependencies: Reasoning and discovery. *ACM Transactions on Database Systems (TODS)* 36, 3 (2011), 1–41.

[83] Shaoxu Song, Lei Chen, and Hong Cheng. 2013. Efficient determination of distance thresholds for differential dependencies. *IEEE Transactions on knowledge and data engineering* 26, 9 (2013), 2179–2192.

[84] Hannu Toivonen et al. 1996. Sampling large databases for association rules. In *Vldb*, Vol. 96. 134–145.

[85] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[86] Jesse Vig. 2019. A multiscale visualization of attention in the transformer model. In *Proceedings of the 57th annual meeting of the association for computational linguistics: system demonstrations*. 37–42.

[87] Jeffrey S Vitter. 1985. Random sampling with a reservoir. *ACM Transactions on Mathematical Software (TOMS)* 11, 1 (1985), 37–57.

[88] Xi Wang, Ruochun Jin, Wanrong Huang, and Yuhua Tang. 2024. Boosting Meaningful Dependency Mining with Clustering and Covariance Analysis. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 639–652.

[89] Catharine Wyss, Chris Giannella, and Edward Robertson. 2001. FastFDs: A heuristic-driven, depth-first algorithm for mining functional dependencies from relation instances extended abstract. In *DaWaK*.

[90] Ying Yan, Liang Jeff Chen, and Zheng Zhang. 2014. Error-bounded sampling for analytics on big sparse data. *Proceedings of the VLDB Endowment* 7, 13 (2014), 1508–1519.

[91] Yunjia Zhang, Zhihan Guo, and Theodoros Rekatsinas. 2020. A statistical perspective on discovering functional dependencies in noisy data. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 861–876.

[92] Yanchang Zhao, Chengqi Zhang, and Shichao Zhang. 2006. Efficient Frequent Itemsets Mining by Sampling. *AMT* 138 (2006), 112–7.

[93] Jinling Zhou, Xingchun Diao, Jianjun Cao, and Zhisong Pan. 2016. An optimization strategy for CFDMiner: an algorithm of discovering constant conditional functional dependencies. *IEICE TRANSACTIONS on Information and Systems* 99, 2 (2016), 537–540.

A TAXONOMY OF CFD PATTERN TUPLES

Conditional functional dependencies (CFDs) extend functional dependencies (FDs) by embedding pattern tuples to enforce data constraints. A pattern tuple t_p is defined over the attribute set $\mathbf{X} \cup \{Y\}$ for a dependency $\mathbf{X} \rightarrow Y$. We categorize CFDs into constant and variable CFDs. This decomposition is supported by Lemma 1 of [29], which shows that any CFD set Σ can be equivalently partitioned into a constant subset Σ_c and a variable subset Σ_v ($\Sigma \equiv \Sigma_c \cup \Sigma_v$).

Constant CFD patterns. A constant CFD pattern tuple assigns a constant value to every attribute, i.e., $t_p[B] = c$ for each $B \in \mathbf{X} \cup \{Y\}$, where $c \in dom(B)$. When the consequent Y is fixed to a constant value, any antecedent attribute in $\mathbf{X}$ assigned to an unnamed variable "_" becomes unnecessary and can be removed without affecting minimality [29]. Consequently, any CFD with a constant RHS can be simplified into a constant form containing only constant attributes in $\mathbf{X} \cup \{Y\}$. Such CFDs represent instance-level rules and are commonly used in data analysis and data cleaning.

Variable CFD patterns. A CFD is a variable CFD if its pattern tuple contains one or more unnamed variables "_". According to the locations of constants and variables, variable CFDs can be categorized into two types. The first type is the pure variable CFD, where $t_p[B] = \_$ for every attribute $B \in \mathbf{X} \cup \{Y\}$. In this case, the CFD reduces to a functional dependency (FD) that holds for all tuples in the relation r. The second type is the mixed variable CFD. In this pattern, the antecedent $\mathbf{X}$ contains both constants and unnamed variables (i.e., $\mathbf{X}^c \neq \emptyset \wedge \mathbf{X}^v \neq \emptyset$), while the consequent remains an unnamed variable ($t_p[Y] = \_$). Unlike pure variable CFDs, mixed variable CFDs describe dependencies that hold only under specific conditions defined by the antecedent constants. For example, the pattern tuple (UK, _ || _) over the dependency (Country, ZipCode $\rightarrow$ City) indicates that the CFD is considered only for tuples where Country = UK.

We note that, for a minimal mixed variable CFD, the consequent $t_p[Y]$ cannot be a constant. If $t_p[Y] = c$, then, according to the semantics of CFDs [29], any antecedent attribute assigned to an unnamed variable "_" no longer contributes additional constraints to the matching tuples. Such attributes therefore become redundant and can be removed without affecting the dependency. As a result, the mixed pattern is simplified into a constant CFD pattern. Hence, to preserve the mixed-variable structure and minimality, the consequent Y must remain an unnamed variable "_".

B EXTENSION AND ADAPTABILITY FOR PATTERN TABLEAU-BASED CFDs

Following Definition 3.3, this section extends SCFDM to support the discovery of tableau-based CFDs ($\varphi_{\mathcal{T}}$). SCFDM can be extended to support pattern tableau with slight modifications, with only changes in PCFDFinder and no changes to the rest of the system.

Impact on AttrFinder. The main concern is whether the attribute correlation extraction step might miss constant relationships (e.g., $\mathbf{X} = A \vee \mathbf{X} = B \rightarrow Y = C$). We clarify that this does not affect AttrFinder. In our Transformer-based design, the self-attention mechanism learns the conditional relationship $P_r(Y|\mathbf{X})$ through masked token reconstruction. If a tableau-based CFD holds in the data, it means that multiple different constant values of $\mathbf{X}$ can strongly predict Y. As a result, during training over the full tuple space, the attention weights will become large for the corresponding $(\mathbf{X}, Y)$ attribute pairs. Therefore, AttrFinder can still correctly capture such correlated attributes and group them into the same correlated set, without requiring any changes to the algorithm.

Impact on RepSampler. Another key concern is whether representative sampling might miss different constant branches needed to build a tableau $\mathcal{T}_p$. We show that RepSampler addresses this issue through its diversity-first design. Specifically, locality-sensitive hashing (LSH) groups tuples into different buckets based on their similarity in the embedding space. Tuples satisfying $\mathbf{X} = A$ and those satisfying $\mathbf{X} = B$ tend to fall into different buckets. By sampling tuples across all non-empty buckets, RepSampler ensures that all groups are fairly represented, including those corresponding to different constant patterns. As a result, the sampled data preserves the tuple diversity needed for tableau construction, without requiring any changes to the algorithm.

Adaptation for PCFDFinder. PCFDFinder can be extended to support tableau-based CFDs in two simple ways. A straightforward approach is to keep existing single-pattern CFD miners (e.g., CFDMiner) and add a post-processing step after parallel mining. In this step, CFDs that share the same attribute set $\mathbf{X} \rightarrow Y$ are merged into a single pattern tableau $\mathcal{T}_p$. This only introduces linear overhead, i.e., $O(|\Sigma|)$, where $|\Sigma|$ is the number of discovered CFDs. Alternatively, the base serial miner can be directly replaced with any tableau-based discovery method, such as those in [40].

Incorporating pattern tableau into existing discovery algorithms leads to an exponential increase in the search space, making the problem intractable for large-scale relations. This challenge further highlights the advantage of SCFDM. By applying AttrFinder, SCFDM partitions the original relation into smaller sub-tables of highly correlated attributes, significantly reducing the search space before mining. Together with the linear-time $O(|\Sigma|)$ post-processing step, SCFDM makes the discovery of tableau-based CFDs both efficient and scalable on large datasets.

C PROOF OF THEOREM 1 AND THEOREM 2

We prove Theorem 1 and Theorem 2 as follows.

C.1 Proof of Theorem 1

The ratio r_c of the variance-augmented bound to Chebyshev's bound can be derived as:

$$r_c = \frac{\frac{\hat{\sigma}(1-\hat{\sigma})\ln(\frac{2}{1-\delta\beta})}{2\epsilon^2}}{\frac{\hat{\sigma}(1-\hat{\sigma})}{\epsilon^2(1-\delta\beta)}} = \frac{(1-\delta\beta)\ln(\frac{2}{1-\delta\beta})}{2}.$$

Since $0 < \delta\beta \leq 1$, the function $f(u) = u\ln(2/u)$ (where $u = 1-\delta\beta$) reaches its maximum near $u = 0.73$ with $f(u) \approx 0.74 < 2$. Thus, $0 < r_c < 1$, proving it is tighter than Chebyshev's bound.

Similarly, to analyze the ratio r_b relative to Bennett's bound, we leverage the inequality $\ln(1+x) > \frac{2x}{2+x}$ and the Taylor expansion

$(\hat{\sigma}+\epsilon)\ln(1+\frac{\epsilon}{\hat{\sigma}})-\epsilon \approx \frac{\epsilon^2}{2\hat{\sigma}}$ for small ϵ, yielding:

$$r_b = \frac{\hat{\sigma}(1-\hat{\sigma})\ln(\frac{2}{1-\delta\beta})}{2\epsilon^2} \cdot \frac{(\hat{\sigma}+\epsilon)\ln(1+\frac{\epsilon}{\hat{\sigma}})-\epsilon}{\ln(\frac{2}{1-\delta\beta})}$$
$$\approx \frac{\hat{\sigma}(1-\hat{\sigma})}{2\epsilon^2} \cdot \frac{\epsilon^2}{2\hat{\sigma}} = \frac{1-\hat{\sigma}}{4} < 1.$$

This concludes that $0 < r_b < 1$, confirming it is tighter than Bennett's bound. □

C.2 Proof of Theorem 2

The proof consists of the following three parts.

(1) Size of samples. The ratio r of the variance-augmented bound to the hybrid Chernoff bound [16] is expressed as:

$$r = \frac{\frac{\hat{\sigma}(1-\hat{\sigma})\ln(\frac{1-\delta\beta}{2})}{-2\epsilon^2}}{\frac{3\tau}{\epsilon^2}\ln(\frac{2}{1-\delta\beta})} = \frac{\hat{\sigma}(1-\hat{\sigma})}{6\tau}.$$

The regime classification boundaries $\hat{\sigma}_1$ and $\hat{\sigma}_2$ are determined by solving the threshold equation $\frac{\hat{\sigma}(1-\hat{\sigma})}{6\tau} = 1$, which yields:

$$\hat{\sigma}_1 = \frac{1-\sqrt{1-24\tau}}{2}, \quad \hat{\sigma}_2 = \frac{1+\sqrt{1-24\tau}}{2},$$

for $\tau \le \frac{1}{24}$. The condition $\frac{\hat{\sigma}(1-\hat{\sigma})}{6\tau} < 1$ ensures that the variance-augmented Hoeffding-Bernstein inequality provides a tighter lower bound. Thus, the bound $N = \frac{\hat{\sigma}(1-\hat{\sigma})\ln(\frac{2}{1-\delta\beta})}{2\epsilon^2}$ holds for the following intervals respectively:

$$\begin{cases} 0 \le \hat{\sigma} \le \hat{\sigma}_1 \text{ and } \hat{\sigma}_2 < \hat{\sigma} \le 1, & \text{when } \tau \le \frac{1}{24} \\ 0 \le \hat{\sigma} \le 1, & \text{when } \tau > \frac{1}{24}. \end{cases}$$

Otherwise, the hybrid Chernoff bound $N = \frac{3\tau}{\epsilon^2}\ln(\frac{2}{1-\delta\beta})$ is selected.

(2) Support threshold. Under random sampling, the occurrence count of any pattern in the sample follows a standard sampling distribution (e.g., hypergeometric or binomial) determined by the original relation r [76]. As a result, the normalized support $\hat{\sigma}'$ measured in the sample relation r_s provides an unbiased estimate of the true support $\hat{\sigma}$ in the full relation r. In other words, random sampling preserves the expected support of a pattern, which can be formally expressed as:

$$\mathbb{E}[\hat{\sigma}'] = \hat{\sigma}.$$

This result implies that, under the additive estimation error ϵ established in Part (1), the sample support $\hat{\sigma}'$ is highly likely to remain close to the true support $\hat{\sigma}$. Specifically, the probability that their difference exceeds ϵ is bounded by $1-\delta$. Therefore, using $\hat{\sigma}'$ as an estimate of $\hat{\sigma}$ introduces no systematic bias and preserves the validity of the subsequent rule discovery process.

(3) Confidence threshold. To ensure that a constraint discovered in the sample relation r_s with confidence δ' still satisfies the target confidence δ in the original relation r, we compensate for the sampling error. Let P denote the true confidence of $\mathbf{X} \to Y$ in r, and let P_s denote the confidence observed in r_s. According to the error bound derived above, the difference between P_s and P is bounded by ϵ with probability at least δ, i.e., $\mathbb{P}(|P_s - P| \le \epsilon) \ge \delta$.

To account for the worst-case effect of sampling noise, we analyze how the confidence estimated from the sample may deviate from its true value in the original relation. Under the Bernoulli sampling model [74], the variance of the empirical confidence estimator is given by $\sigma^2_{sample} = \frac{P(1-P)}{N}$, where P is the true confidence and N is the sample size. Let $\Delta P = P_s - P$ denote the estimation error introduced by sampling. Since Part (1) guarantees that $|\Delta P| \le \epsilon$, we further examine how this bounded error affects the confidence threshold. For small values of ϵ, a first-order Taylor approximation [37] provides an accurate estimate of this effect. Specifically, we expand the confidence evaluation function around the threshold δ as:

$$f(P_s) \approx f(P) + \frac{\partial f}{\partial P}\Delta P + O(\Delta P^2).$$

To account for the worst-case impact of sampling noise, we derive a mapping from the target confidence threshold δ to the adjusted sample confidence threshold δ', given by:

$$\delta = \delta' \cdot \left(1 + \frac{\epsilon}{2}Var(\mathbf{X})\right) \approx \delta' \cdot \frac{1+\frac{\epsilon}{4}}{1-\frac{\epsilon}{4}},$$

where the maximum variance component $Var(\mathbf{X})$ under random assignment non-trivially approaches the tightly bounded constant scaling of $\frac{1}{2}$. Rearranging the algebraic terms to isolate the conservative sample threshold δ', we obtain:

$$\delta' = \frac{1-\frac{\epsilon}{4}}{1+\frac{\epsilon}{4}}\delta.$$

This calibration depends only on the statistical properties of sampling error and is independent of the specific CFD discovery strategy (e.g., BFS-based lattice traversal). As a result, it can be applied universally across different CFD discovery algorithms. □

D TRAINING DETAILS IN AttrFinder

The training of AttrFinder is a self-supervised process designed to extract dependencies through masked reconstruction over the tensor $\mathcal{T}$. Given a batch of B embedded tuples $\mathcal{T}_{\mathcal{B}} \in \mathbb{R}^{B\times m\times d}$, where $B \le n$, the training procedure follows four critical phases.

Stochastic attribute masking. To compel the model to learn conditional dependencies $P(Y|\mathbf{X})$, we apply a masking operator $\tilde{\mathcal{T}}_{\mathcal{B}} = \text{Mask}(\mathcal{T}_{\mathcal{B}})$. For each tuple t_j in the batch, we randomly select a subset of attributes and replace their d-dimensional embeddings $\mathbf{E}_i^{(j)}$ with a learnable $[MASK]$ token. This simulates the discovery of unknown consequents at scale, forcing the model to reconstruct the masked attributes by attending to the remaining contextual attributes across diverse tuple instances.

Multi-Head attention interaction (Encoder). The masked batch $\tilde{\mathcal{T}}_{\mathcal{B}}$ is processed through L layers. In each layer $l \in [1, L]$, the self-attention mechanism computes interaction matrices for each tuple t_j:

$$\mathbf{A}_{l,h}^{(j)} = \text{softmax}\left(\frac{Q_{l,h}^{(j)}(K_{l,h}^{(j)})^T}{\sqrt{d_k}}\right),$$

where Q, K are linear projections of the j-th tuple's representation from the previous layer, and $\mathbf{A}_{l,h}^{(j)} \in \mathbb{R}^{m\times m}$ captures the tuple-specific correlations.

Segmented categorical reconstruction. The final hidden state of the L-th layer, $\mathbf{H}^{(L)} \in \mathbb{R}^{m\times d}$, is passed through a linear projection layer. For each masked attribute A_i with c_i categories, we

apply a segmented softmax to reconstruct the original zero-one distribution:

$$\hat{y}_{i,k} = \frac{\exp(W_{out}^{(i)}\mathbf{h}_i^{(L)} + b_{out}^{(i)})_k}{\sum_{p=1}^{c_i}\exp(W_{out}^{(i)}\mathbf{h}_i^{(L)} + b_{out}^{(i)})_p}.$$

This ensures the output for each attribute segment represents a probability distribution, where $\hat{y}_{i,k}$ approximates the conditional probability $P(A_i = v_k \mid attr(\mathcal{R}) \setminus \{A_i\})$.

Optimization and convergence. The model parameters $\Theta = \{\Theta_{emb}, \Theta_{attn}, \Theta_{out}\}$ are optimized end-to-end. Specifically, the embedding parameters $\Theta_{emb} = \{W_{emb}^{(i)}, \mathbf{b}_{emb}^{(i)}\}_{i=1}^{m}$ are updated simultaneously with the attention and projection weights. We minimize the batch-averaged cross entropy loss $\mathcal{L}$ aggregated over all masked positions across B tuples:

$$\mathcal{L} = -\frac{1}{B}\sum_{j=1}^{B}\sum_{i\in\text{Masked}_j}\sum_{k=1}^{c_i} y_{i,k}^{(j)}\log(\hat{y}_{i,k}^{(j)}),$$

where $y_{i,k}^{(j)}$ is the ground-truth label for the i-th attribute of the j-th tuple. This global objective ensures that the gradients $\nabla_\Theta\mathcal{L}$ reflect the statistical consensus of the entire relation r. During each training step, the gradients are backpropagated from the reconstruction layer through the Transformer layers to the attribute embedding layer. The update rule for the embedding parameters at iteration t using the Adam optimizer is:

$$\Theta_{emb}^{(t+1)} = \Theta_{emb}^{(t)} - \eta\cdot\text{Adam}(\nabla_{\Theta_{emb}}\mathcal{L}),$$

where η is the learning rate (e.g., 5×10^{-4}). This optimization ensures that the dense representations $\mathbf{E}_i$ in the latent space are tuned to minimize uncertainty in attribute reconstruction. Through these iterative updates, the attention maps $\mathbf{A}$ and the embeddings Θ_{emb} are updated, eventually converging to a state that reflects the underlying conditional functional dependencies of the relation r.

E COMPLEXITY OF AttrFinder

To evaluate the scalability of AttrFinder, we analyze its computational complexity across three main stages and compare it with traditional discovery methods. This comparison highlights the efficiency advantages of our neural approach.

Data Encoding. The first stage converts the relation r into a tensor representation $\mathcal{T} \in \mathbb{R}^{n\times m\times d}$. Scanning the n tuples to identify distinct values and perform one-hot encoding requires $O(n\cdot m)$. The resulting sparse vectors are then projected into a d-dimensional embedding space, which also takes $O(n\cdot m)$ when treating d as a constant. Therefore, the overall complexity of this stage is linear in the size of the relation, making it efficient for large-scale datasets.

Transformer-based learning. This is the most computationally intensive phase, where the model captures attribute correlations across the dataset. For each of the n tuples, the self-attention mechanism computes an $m\times m$ interaction matrix. Across all epochs and layers, the training complexity is $O(n\cdot m^2)$ when treating the number of epochs, layers, and embedding dimension as constants. Although these constants affect absolute runtime, the growth remains quadratic in the number of attributes m, far more efficient than the exponential $O(2^m)$ cost of traditional search-based methods. Moreover, the operations are highly parallelizable on GPUs.

Correlated attribute set extraction. Once the model has converged, CFD extraction becomes largely independent of the dataset size n. The first step computes the saliency matrix $\bar{\mathbf{A}}$ by aggregating attention maps across all n tuples and L Transformer layers, resulting in a complexity of $O(n\cdot m^2)$ when L and the number of attention heads H are treated as constants. The second step scans $\bar{\mathbf{A}}$ to identify candidate antecedent sets $\mathbf{X}$ for each of the m possible consequents, requiring only $O(m^2)$ time.

F ANALYSIS OF TRANSFORMER TRAINING OVERHEAD IN AttrFinder

To better understand the runtime of SCFDM, this section reports the Transformer training time and its proportion in the total runtime.

Constant CFD discovery ($\text{SCFDM}_{\text{part}}$). For constant CFD discovery tasks, the main cost is learning attribute correlations. The Transformer training time ranges from 6.32s to 20.68s across benchmarks, with an average of 11.43s. On average, it accounts for 29.07% of the total runtime. The highest proportion appears on Flight, where training takes 6.32s of 10.98s total runtime (57.56%), followed by RT (54.14%, 7.84s / 14.48s), Crop (52.03%, 8.22s / 15.80s), and Adult (49.97%, 9.96s / 19.93s). As dataset size or tuple diversity increases, the rule discovery phase takes a larger share of runtime, reducing the relative cost of Transformer training. This is reflected in Census (37.06%, 12.33s / 33.27s), Syn-2 (18.54%, 14.64s / 78.96s), and Syn-1 (20.30%, 20.68s / 101.85s), where Transformer training occupies a smaller proportion of the total runtime.

Both constant and variable CFD discovery ($\text{SCFDM}_{\text{all}}$). In both constant and variable discovery tasks, the search space grows exponentially, so the Transformer training overhead is amortized by the much larger search cost, reducing its average runtime share to 0.54% across datasets. It accounts for a maximum proportion of 10.98% on the Adult dataset, where it requires 10.12s out of the 92.18s total execution time, followed by 5.49% on Flight (6.55s out of 119.32s). Crucially, on larger datasets or those with higher tuple diversity, where traditional rule discovery algorithms face the most severe execution bottlenecks, the proportion of Transformer training time decreases significantly. This empirical trend is validated across the remaining datasets, where the training consumes 9.82s out of 744.68s on Census (1.32%), 15.43s out of 3,392.30s on Syn-2 (0.45%), 8.48s out of 2,031.53s on RT (0.42%), 18.15s out of 4,682.95s on Syn-1 (0.39%), and reaches its minimum proportion of 0.28% on the Crop dataset, taking 9.16s out of the 3,293.83s total end-to-end time. This scaling behavior demonstrates the scalability and efficiency of SCFDM in handling large search spaces, showing that the Transformer training delivers performance gains as data size and search space increase.

G COMPLEXITY OF RepSampler

To evaluate the efficiency of RepSampler, we analyze its computational complexity across its main operational stages and show that it scales linearly with the number of tuples n.

Hash-based partitioning. This phase organizes the data into structural bucket without the overhead of quadratic comparisons. RepSampler employs an LSH framework based on the MinHash strategy to map n tuples into similarity buckets. Since the algorithm

Table 7: Effectiveness evaluation on constant CFDs

Dataset	Metric	CFDMiner	$\text{SCFDM}_{\text{part}}$
RT	Precision	0.980	**0.988**
	Recall	**0.971**	**0.971**
	F_1	0.975	**0.979**
Census	Precision	0.942	**0.985**
	Recall	**0.977**	0.969
	F_1	0.959	**0.977**
Crop	Precision	0.970	**0.984**
	Recall	**0.965**	0.946
	F_1	**0.967**	0.965
Flight	Precision	0.994	**0.999**
	Recall	**0.996**	**0.996**
	F_1	0.995	**0.997**
Adult	Precision	**1.000**	**1.000**
	Recall	**1.000**	**1.000**
	F_1	**1.000**	**1.000**
Syn-1	Precision	0.967	**0.981**
	Recall	0.970	**0.964**
	F_1	0.968	**0.972**
Syn-2	Precision	0.980	**0.992**
	Recall	**0.992**	0.990
	F_1	0.986	**0.991**

only processes the m non-zero entries (original attributes) for each of the hash functions, the complexity of this stage is $O(n)$. Unlike iterative clustering (e.g., K-means [60]) or density-based methods (e.g., DBSCAN [25]), which often scale quadratically or require multiple passes, our hashing-based approach achieves partitioning in a single linear scan, effectively bypassing the $O(n^2)$ bottleneck of pairwise distance calculations.

Bucket management. This stage is designed to preserve rare patterns that still represent valid CFDs, even though they have low support. Sorting the k resulting buckets by their population size takes $O(k \log k)$ to facilitate the diversity-first selection logic. In real-world datasets, the number of buckets k is constrained by the LSH parameters and is significantly smaller than the total number of tuples ($k \ll n$), making this overhead negligible while ensuring the sampling is not biased toward dominant, redundant patterns.

Stratified sampling. The final phase extracts the minimal set of tuples required to satisfy the discovery recall guarantee. RepSampler iteratively extracts representative tuples from the managed buckets until the theoretical sampling lower bound N is reached, resulting in $O(N)$. While traditional discovery algorithms must process the full dataset n, RepSampler reduces the input size to a compact representative set N. Since $N \ll n$, RepSampler lowers the computational cost of downstream mining while maintaining a recall guarantee.

H ADDITIONAL EXPERIMENTAL DETAILS

H.1 Evaluation on Constant CFDs in Exp-3.

To avoid evaluation bias caused by CFDMiner's restriction to constant CFDs, we conduct an additional evaluation focusing on constant CFD discovery. Under this setting, the ground truth contains only constant CFDs, which allows a direct comparison between CFDMiner and $\text{SCFDM}_{\text{part}}$.

As reported in Table 7, even when restricted only to the constant CFDs, $\text{SCFDM}_{\text{part}}$ consistently achieves superior F_1-scores across most evaluated datasets, with an average F_1-score of 0.980 and a peak value of 1.000. In contrast, CFDMiner achieves an average F_1-score of 0.976 across these datasets. While CFDMiner achieves higher recall than in both constant and variable CFD discovery setting (Table 4), it still exhibits low precision and F_1 scores on large-scale datasets (e.g., Census). Specifically, on the large Census dataset with diverse tuples, CFDMiner achieves an F_1-score of only 0.959 due to low precision, whereas $\text{SCFDM}_{\text{part}}$ improves it to 0.977. CFDMiner discovers a large number of spurious CFDs that hold only by coincidence rather than reflecting true attribute correlations. In comparison, $\text{SCFDM}_{\text{part}}$ identifies correlated attributes and achieves high precision and F_1-score in CFD discovery.

The effectiveness of $\text{SCFDM}_{\text{part}}$ in discovering constant CFDs stems from its ability to extract correlated attribute sets, apply representative tuple sampling, and perform parallel CFD discovery on sub-tables, which reduces the search space and improves the rule quality of constant CFD discovery.

H.2 Parameter Sensitivity Results on the Remaining Five Datasets in Exp-4.

To supplement the parameter sensitivity evaluation presented in Exp-4, we report the complete experimental results regarding the impact of normalized support thresholds $|\hat{\sigma}|$ on the remaining five datasets (Census, Crop, Flight, Adult, and Syn-2) in Table 8.

The experimental results across these five datasets demonstrate a performance trend highly consistent with the findings observed on the RT and Syn-1 datasets in Exp-4. For the relatively simple datasets, (e.g., Flight and Adult), all baselines (CTANE, Itemset-First, and FD-First) complete their execution within the 9-hour time limit across support thresholds from 2×10^{-3} to 5×10^{-3}, thus matching the optimal F_1-score achieved by SCFDM.

In contrast, on datasets with high tuple diversity or large scale, including Census, Crop, and Syn-2, the performance of all baselines degrades significantly when the support threshold is configured to a low value (e.g., $|\hat{\sigma}| = 10^{-3}$). Specifically, under $|\hat{\sigma}| = 10^{-3}$, the F_1-scores of CTANE drop to 0.663 on Census, 0.588 on Crop, and 0.472 on Syn-2 due to time-limit termination. Conversely, SCFDM bypasses these computational bottlenecks by extracting correlated attribute sets and applying parallel CFD discovery on row-sampled sub-tables, maintaining high F_1-scores (above 0.89) across all settings and outperforming baselines in low-support scenarios.

H.3 Impact of Maximum CFD Size and Heuristic Optimization

The worst-case runtime of CFD discovery is affected by the maximum CFD size $|k|$, as the search space grows exponentially with $\binom{m}{|k|}$. Instead of using a fixed limit on $|k|$, SCFDM adopts a heuristic strategy to determine $|k|$.

Specifically, our framework determines $|k|$ from the weight distribution in the global attention map $\bar{\mathbf{A}}$. Instead of a fixed constraint on the maximum rule length $|k|$, the retention factor γ flexibly controls the maximum rule length k_Y for each attribute Y. For each consequent Y, SCFDM ranks candidate antecedents in descending order of attention weights $\bar{\mathbf{A}}[Y, \cdot]$. The adaptive CFD size limit k_Y is then defined as the smallest number of top-ranked attributes whose cumulative attention mass reaches $\gamma \in (0, 1]$, i.e., $\sum_{i=1}^{k_Y} \bar{\mathbf{A}}[Y, i] \geq \gamma$.

To ensure a theoretically grounded and automatic initialization

of the hyperparameter γ, we derive a robust formulation for selecting its optimal value, inspired by cumulative variance criteria in dimensionality reduction [58, 86]:

$$\gamma = \max\left(\gamma_0, 1 - \frac{1}{m}\sum_{i=1}^{m}\mathcal{H}(\bar{\mathbf{A}}[A_i, \cdot])\right),$$

where $\gamma_0 = 0.85$ represents the baseline information density, and m is the total number of attributes. Specifically, the normalized Shannon entropy [79] for each attention row vector is defined as:

$$\mathcal{H}(\bar{\mathbf{A}}[Y, \cdot]) = -\frac{1}{\ln m}\sum_{k=1}^{m}\bar{\mathbf{A}}[Y, k]\ln\left(\bar{\mathbf{A}}[Y, k] + \epsilon_0\right),$$

where $\epsilon_0 = 10^{-5}$ is a tiny smoothing constant to prevent numerical underflow for zero entries. A low-entropy (highly concentrated) attention distribution indicates that a small number of antecedents carry most of the dependency signal, which increases γ and results in stricter selection that retains only strongly supported antecedents. Conversely, a high-entropy (near-uniform) distribution indicates that attention is spread more evenly across candidates, which decreases γ and restricts the antecedent set size, reducing combinatorial explosion under uncertain dependency patterns.

Our empirical evaluations across real-world relations (e.g., the Adult and Crop datasets) validate the robustness and effectiveness of this heuristic strategy. In our experiments, adjusting the retention factor γ via our formulation within the range of $[0.85, 0.95]$ consistently enables the framework to dynamically bound k_Y between 2 and 7 across diverse attribute schemas. The empirical results on the Crop dataset under the default setting show that this heuristic strategy, which adaptively determines a per-attribute limit k_Y instead of using a fixed limit k (e.g., $k = 6$) for all attributes, achieves a better balance between efficiency and discovery accuracy. It reduces the runtime from 1,724.2s to 655.4s and improves the F_1-score from 98.0% to 98.2%. This data-driven filtering prunes unpromising attribute combinations, proving that our heuristic setting of k_Y reduces search space while maintaining discovery accuracy.

H.4 Systematic Guidance for Hyperparameter Optimization

To provide a workflow for deploying SCFDM, this section presents a hyperparameter optimization guide. The configuration of SCFDM is divided into two categories: (1) *discovery guarantee parameters*, which control sampling bounds; and (2) *structural partitioning parameters*, which determine correlated attribute sets.

Category 1: Discovery guarantee parameters ($\epsilon, \tau, \beta, \hat{\sigma}, \delta$). These parameters directly determine the sample size N in Theorem 2. Specifically, the normalized support $\hat{\sigma}$ and confidence δ are directly specified by the user to define the rule discovery criteria and mining granularity, with typical empirical values set to $\hat{\sigma} = 2\times10^{-3}$ and $\delta = 0.95$. For the remaining parameters, we recommend a sequential configuration starting with the recall parameter β. As the core control for rule preservation, β defines the sampling bounds and is typically set to $\beta \geq 0.90$ in data cleaning tasks to reduce the risk of missing CFDs. Subsequently, the additive support error ϵ is tuned, which increases the sample size N at rate $O(1/\epsilon^2)$. Users should initialize $\epsilon = 5 \times 10^{-4}$; for datasets with attribute count $m > 50$, it can be reduced to 2×10^{-4} to increase the sample size and better preserve rare tuple patterns. Finally, the Chernoff ratio τ controls the selection of N in Theorem 2. Setting $\tau = 0.05$ by default provides tight sample bounds when the normalized support threshold $\hat{\sigma} \leq 0.14$. For relations with $\hat{\sigma} \leq 0.02$, τ can be reduced to 0.02 to enforce a tighter sampling.

Category 2: Structural partitioning parameters (γ, θ_{att}). These parameters dictate the pruning threshold within the correlation extraction stage (Section 6.3) and directly manipulate the sub-table candidate space. Their adjustment starts with the automation of the attention retention factor γ. Instead of setting a fixed threshold, users should compute γ following the formulation in Appendix H.3 with the base density parameter $\gamma_0 = 0.85$, which dynamically balances the local size k_Y (typically between 2 and 7). Subsequently, the scaling factor z in the significance threshold $\theta_{att} = z/m$ is adaptively formulated as: $z = \frac{1}{1-\bar{\mathcal{H}}+\epsilon_1}$, where $\bar{\mathcal{H}} = \frac{1}{m}\sum_{i=1}^{m}\mathcal{H}(\bar{\mathbf{A}}[A_i, \cdot])$ represents the global mean normalized Shannon entropy [79], and $\epsilon_1 = 10^{-5}$ is a numerical stability constant. On clean datasets with clear dependency signals, the system has low entropy ($\bar{\mathcal{H}} \to 0$), and z is reduced toward 1. As a result, the pruning threshold θ_{att} converges to the baseline $1/m$, preserving attributes that carry useful signals. In contrast, when noise is high and attention is widely spread ($\bar{\mathcal{H}} \to 1$), z increases, which raises θ_{att} and filters out weak candidate combinations. This prevents the search space from growing excessively without requiring fixed thresholds (θ_{att}).

Step-by-step optimization pipeline. For a new dataset, the operational sequence is executed as follows. First, the normalized support $\hat{\sigma}$ and confidence δ are user-specified parameters that define the rule discovery problem. The formal problem statement is given in Section 3 (p. 4). Based on our empirical settings, we initialize these sampling bounds with the default values $\hat{\sigma} = 2 \times 10^{-3}$, $\beta = 0.9$, $\delta = 0.95$. We set $\tau = 0.05$ by default, and reduce it to 0.02 if the normalized support threshold $\hat{\sigma} \leq 0.02$. We then set $\epsilon = 5 \times 10^{-4}$, and reduce it to 2×10^{-4} if the attribute count $m > 50$. Second, run preview profiling on a 5%–20% data slice to extract the global attention map $\bar{\mathbf{A}}$ and compute the mean entropy $\bar{\mathcal{H}}$. Third, resolve γ and z directly from $\bar{\mathcal{H}}$. For example, on a dataset with $\bar{\mathcal{H}} = 0.20$, the formulation yields $z = 1.25$, preserving more candidate attributes. Conversely, on a noisy dataset with $\bar{\mathcal{H}} = 0.80$, it yields $z = 5.00$, leading to stronger pruning of the sub-table space.

H.5 Computation reduction of AttrFinder

The computation reduction in AttrFinder is the ratio of the lattice traversal computational overhead in the sub-tables to that of the original instance r, excluding representative sampling and pruning. It can be expressed as: $(\sum_{i=1}^{l} m_i \times d^{m_i})/(m \times d^m)$, where m is the number of attributes in r, d is the average domain size, l is the number of sub-tables, and m_i is the number of attributes in the i-th sub-table. In the experiment, AttrFinder identified 85, 45, 175, 25, 14, 48, and 37 correlated vector sets on the RT, Census, Crop, Flight, Adult, Syn-1, and Syn-2 datasets, respectively. Compared to direct mining, this leads to significant computational overhead reductions of at least $\frac{40}{d^{45}_{\text{RT}}}$, $\frac{12}{d^{33}_{\text{Census}}}$, $\frac{70}{d^{105}_{\text{Crop}}}$, $\frac{11}{d^{14}_{\text{Flight}}}$, $\frac{8}{d^{6}_{\text{Adult}}}$, $\frac{20}{d^{28}_{\text{Syn-1}}}$, and $\frac{19}{d^{18}_{\text{Syn-2}}}$ on these datasets (calculated based on the largest sub-table).

Table 8: Parameter sensitivity results (support threshold $|\hat{\sigma}|$) on the remaining five datasets in Exp-4

Dataset	Algorithm	Metric	1×10^{-3}	2×10^{-3}	3×10^{-3}	4×10^{-3}	5×10^{-3}
Census	CFDMiner	F_1	0.103	0.117	0.145	0.160	0.188
	CTANE		0.663	0.971	**0.984**	0.979	**0.988**
	Itemset-First		0.755	0.973	0.981	**0.986**	0.983
	FD-First		0.712	0.978	0.981	0.983	0.983
	SCFDM		**0.919**	**0.985**	0.982	**0.986**	**0.988**
Crop	CFDMiner	F_1	0.094	0.155	0.157	0.168	0.172
	CTANE		0.588	0.950	0.958	0.964	**0.975**
	Itemset-First		0.674	0.949	0.959	0.968	0.972
	FD-First		0.695	0.949	0.970	**0.970**	0.970
	SCFDM		**0.898**	**0.952**	**0.974**	**0.970**	0.972
Flight	CFDMiner	F_1	0.140	0.246	0.233	0.251	0.256
	CTANE		0.863	**0.997**	0.996	0.993	0.996
	Itemset-First		0.907	**0.997**	0.997	0.995	0.996
	FD-First		0.924	**0.997**	0.996	**0.998**	**0.998**
	SCFDM		**0.994**	**0.997**	**0.998**	0.995	**0.998**
Adult	CFDMiner	F_1	0.165	0.505	0.512	0.537	0.565
	CTANE		0.988	**1.000**	**1.000**	0.996	**1.000**
	Itemset-First		0.992	**1.000**	0.998	0.997	0.998
	FD-First		**0.994**	**1.000**	0.994	0.995	**1.000**
	SCFDM		**0.994**	**1.000**	0.997	**1.000**	**1.000**
Syn-2	CFDMiner	F_1	0.087	0.146	0.166	0.174	0.178
	CTANE		0.472	0.970	0.970	0.975	**0.990**
	Itemset-First		0.504	0.967	0.968	0.977	0.983
	FD-First		0.521	0.965	0.970	0.980	0.983
	SCFDM		**0.931**	**0.973**	**0.972**	**0.985**	**0.990**

* Results obtained before reaching the resource limit (9 hours of time or 224GB of memory).
Bold indicates the best performance.

H.6 Computation reduction of RepSampler

The computation reduction in RepSampler is the ratio of the number of tuples in the sample (n_s) to the total number of tuples in the original relation (n), i.e., n_s/n. RepSampler selected 8,519, 10,544, 16,063, 41,057, 8,422, 8,408, and 8,397 representative tuples from the RT, Census, Crop, Flight, Adult, Syn-1, and Syn-2 datasets, respectively. This selection yielded computation reductions of 0.069, 0.053, 0.049, 0.041, 0.177, 0.420, and 0.420, respectively. Notably, the computational overhead exhibits a linear decrease as the sample size n_s is reduced.

H.7 Details of Experimental Real-world Datasets

We provide a comprehensive overview of the five real world datasets used in our experiments, focusing on their structural characteristics and relevance to CFD discovery.

(1) RT-IOT (RT). The RT-IOT-2022 dataset is a comprehensive resource derived from a real time IOT infrastructure [80]. Unlike traditional static datasets, it integrates a diverse range of physical IOT devices including ThingSpeak LED, Wipro Bulb, and MQTT Temp alongside sophisticated network attack methodologies. The dataset is designed to provide a general representation of real world scenarios by capturing both normal operations and adversarial behaviors. To ensure high fidelity in network traffic analysis, the bidirectional attributes of the traffic were meticulously captured using the Zeek network monitoring tool and the Flowmeter plugin. This process resulted in a rich feature set that allows researchers to analyze complex traffic patterns across various protocols.

The dataset contains a total of 123,117 instances, categorized into attack patterns and normal patterns as shown in Table 9. In our research, this dataset serves as a benchmark for mining CFDs. Specifically, the coexistence of normal and malicious traffic allows us to identify dependencies that hold under normal conditions but are violated during specific attack phases. For example, a CFD may capture the stable relationship between a device type and its expected traffic volume, which becomes an essential constraint for identifying anomalous behavior in IOT environments.

(2) Census-Income (Census). The Census-Income dataset consists of weighted census records extracted from the 1994 and 1995 Current Population Surveys conducted by the U.S. Census Bureau. This dataset is a high dimensional resource containing 41 demographic and employment related variables for each individual, including features such as age, class of worker, education, and marital status. It provides a total of 199,523 training instances and 99,762 test instances, which were split in approximately a two thirds to one third proportion. A critical attribute of this dataset is the instance weight, which indicates the number of people in the actual population that each record represents due to stratified sampling. While this field is essential for deriving statistical conclusions about the general population, it is typically excluded from the feature set used in classification models. The primary task associated with this dataset is a binary classification problem. Similar to the original Adult database, incomes have been binned at the 50,000 dollar level. However, the target variable in this specific version was derived from the total person income field rather than adjusted gross income, which introduces distinct statistical characteristics.

For the purpose of mining CFDs, the Census Income dataset is particularly valuable due to its rich set of categorical and numerical attributes. It allows for the discovery of complex dependencies between demographic profiles and socio economic status, such as the relationship between occupation codes and wage per hour conditioned on specific industry sectors.

(3) Crop mapping using fused optical radar data (Crop). The Crop dataset is a fused bi temporal resource designed for sophisticated cropland classification near Winnipeg, Manitoba, Canada. Collected in 2012, this dataset combines optical imagery from RapidEye satellites with Polarimetric Synthetic Aperture Radar (PolSAR) data from the Unmanned Aerial Vehicle Synthetic Aperture Radar (UAVSAR) system. By integrating these complementary sensing modalities, the dataset captures a significant number of temporal, spectral, textural, and polarimetric features that are essential for distinguishing between various crop types.

The dataset contains 175 attributes in total, structured around two specific observation dates: July 05 and July 14, 2012. The feature space is divided into several categories: (1) Polarimetric features: 49 radar features per date (f1 to f98) capturing scattering properties such as sigHH, sigVV, and complex decompositions like Cloude Pottier parameters (Entropy, Anisotropy) and Freeman Durden components. (2) Optical features: 38 optical features per date (f99 to f174) including spectral bands (Blue, Green, Red, Red edge, NIR) and derived vegetation indices such as NDVI, EVI, and SAVI, alongside textural metrics derived from Principal Component Analysis. There are seven distinct crop type classes identified in this dataset: Corn, Peas, Canola, Soybeans, Oats, Wheat, and Broadleaf.

In our study, the Crop dataset serves as a benchmark for mining CFDs across multi modal sensor data. The rich set of 175 attributes allows for the discovery of dependencies where spectral signatures

Table 9: Distribution of attack and normal patterns in the RT-IOT dataset.

Attack Patterns	Count	Normal Patterns	Count
DOS SYN Hping	94, 659	Amazon Alexa	86, 842
ARP poisioning	7, 750	MQTT	8, 108
NMAP UDP SCAN	2, 590	Thing speak	4, 146
NMAP XMAS TREE SCAN	2, 010	Wipro bulb	253
NMAP OS DETECTION	2, 000		
NMAP TCP scan	1, 002		
DDOS Slowloris	534		
Metasploit Brute Force SSH	37		
NMAP FIN SCAN	28		

from optical sensors are constrained by polarimetric backscatter measurements. For instance, a CFD might define a stable relationship between a specific crop type and its expected NDVI range, conditioned on the stability of its radar backscatter cross section during different growth stages.

(4) 2015 Flight Delays and Cancellations (Flight). The Flight dataset, specifically the 2015 Flight Delays and Cancellations resource, provides an exhaustive record of domestic flight operations within the United States. It encompasses nearly 5.8 million flights, capturing critical metrics such as scheduled and actual departure times, arrival delays, and cancellation reasons. The dataset is structured to facilitate multi dimensional analysis across three primary entities: Airlines (14 major carriers), Airports (322 locations), and Time (year, month, day, and day of week). The raw data underwent a rigorous cleaning process to ensure analytical integrity. This included converting blank entries into NA values, omitting records with missing scheduled time or air time, and merging flight records with airport and airline lookup tables to resolve numeric codes into descriptive names. Statistical profiling reveals that approximately 1.6 percent of all flights were cancelled, with over half of these cancellations attributed to weather conditions (category B), followed by airline related issues (category A) and air system delays (category C).

For CFD discovery, this dataset is particularly valuable due to the intricate relationships between categorical attributes (e.g., Origin Airport, Airline) and numerical performance metrics (e.g., Departure Delay, Taxi In time). It allows for the identification of dependencies where the probability of a delay is constrained by specific temporal or geographical contexts. For example, a CFD might capture a stable constraint where flights operated by a specific carrier from a certain hub consistently maintain a lower taxi out duration, provided the weather conditions remain within normal parameters.

(5) Adult. The Adult dataset was extracted by Barry Becker from the 1994 United States Census database. To ensure data quality, a set of reasonably clean records was filtered based on specific demographic and economic conditions, including individuals older than 16 years, an adjusted gross income exceeding 100 dollars, and a recorded work volume of more than zero hours per week.

The primary prediction task for this dataset is a binary classification problem: determining whether an individual's annual income exceeds the 50,000 dollar threshold. The dataset comprises 14 diverse attributes that capture a holistic view of an individual's socio economic profile: (1) Categorical Attributes: These include workclass (e.g., Private, Federal gov), education level, marital status, occupation, relationship role, race, sex, and native country. (2) Continuous Attributes: These encompass age, final weight (fnlwgt), education num, capital gain, capital loss, and hours per week.

In the context of our research on CFDs, the Adult dataset provides a robust environment for discovering complex logical constraints. Its mix of categorical and continuous variables allows for the identification of dependencies where professional attributes (such as occupation and education) are functionally linked to income levels under specific demographic conditions (such as workclass or age groups). For instance, a CFD might define a high confidence dependency between a doctorate degree and the high income class, specifically within the context of private sector employment.